

TEMPLATE PROJECT WORK

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (<i>pari a circa Minimo 12 – Massimo 20 pagine</i>)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Gabriele Stagnitta
Numero di matricola	0312401645
Tema n. (Indicare il numero del tema scelto):	Tema n. 5
Titolo del tema (Indicare il titolo del tema scelto):	Machine Learning per Processi Aziendali
Traccia del PW n. (Indicare il numero della traccia scelta):	PW 18
Titolo della traccia (Indicare il titolo della traccia scelta):	Triage automatico dei ticket con Machine Learning
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Machine Learning applicato allo Smart Ticketing: dalla generazione dati tramite LLM allo smistamento automatico dei ticket
Extra	Per la consegna finale svilupperò il PW in parallelo a questo documento in un altro file in formato LaTeX, con lo scopo di garantire una maggiore fruibilità del testo e una resa grafica ottimale. Non sarà disponibile nella versione della pre valutazione per mancanza di tempo, ma ci sarà sicuramente nella prossima versione.

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

La realizzazione di questo **Project Work** ha permesso l'integrazione tra le **conoscenze multidisciplinari** maturate nel **percorso di studi** e quelle acquisite in **ambito professionale**. Lavorando come insegnante di coding per studenti tra i 7 e i 18 anni e come responsabile dei corsi di **Python** e di **Intelligenza Artificiale con Python**, ho potuto applicare sia in fase di stesura dei vari **Jupyter Notebook** che nella scrittura di questo documento diverse metodologie pratiche che utilizzo giornalmente per la composizione dei materiali didattici. Inoltre ho potuto

utilizzare alcune competenze trasversali per ottimizzare i processi di sviluppo, implementazione e analisi delle fasi.

Dal **punto di vista accademico**, ho avuto modo di lavorare al progetto in contemporanea alla frequenza del corso di **Programmazione 2** del Prof. Claudio Tommazoli. Questo mi ha permesso di avvalermi delle competenze apprese sull'utilizzo di librerie come *pandas*, *seaborn*, ecc... Difatti, come verrà meglio analizzato in questo documento, ho più volte avuto modo di utilizzare questi strumenti per l'**EDA** (Exploratory Data Analysis). Altra cosa utile appresa nel corso è sicuramente la libreria *Tkinter*, che non conoscevo prima di seguire l'insegnamento, utilizzata per un approccio iniziale per la creazione della Dashboard finale, nonostante alla fine abbia deciso di utilizzare un framework più moderno, ovvero *niceGUI*.

Proprio in merito all'EDA e analisi dei dati visualizzati, ho trovato fondamentali i concetti appresi studiando il corso di **Calcolo delle Probabilità e Statistica** tenuto dal Prof. Sergio Frigeri. Difatti le capacità analitiche apprese nel corso mi hanno permesso di interpretare con maggiore consapevolezza i dati generati e gli output della pipeline ML. Queste conoscenze hanno inoltre permesso di soddisfare la mia curiosità sul funzionamento di alcuni **algoritmi matematici essenziali** di *scikit-learn*, come ad esempio il *TfidfVectorizer* (ragionamento matematico che citerò in questo documento).

Una menzione d'onore va infine al corso di **Algoritmi e Strutture Dati** del Prof. Stefano D'Urso, al corso di **Ingegneria del Software** tenuto dai Docenti Fabiano Pecorelli, Massimiliano Pirani, Roberto Vergallo e Martina Iammarino, e al corso di **Tecnologie Web**, che ho seguito nella mia precedente Università. I primi due insegnamenti mi hanno aiutato a immaginare la struttura del codice non solo come una sequenza di istruzioni, ma come un'architettura software strutturata e stratificata, dove ogni istruzione va ponderata per garantire la massima efficienza e manutenibilità in caso di modifiche durante il ciclo di vita del progetto (e se ne sono presentate non poche). Le competenze acquisite in **Tecnologie Web**, infine, si sono rivelate determinanti nello sviluppo di un'interfaccia utente (UI) moderna e funzionale al progetto.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato
(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

La realizzazione dell'elaborato è stata organizzata in *fasi strutturate* per ottimizzare il workflow: dall'analisi delle procedure per la creazione di un dataset sintetico fino al deploy della web app come risultato finale. I vari Notebook sono stati creati e compilati sequenzialmente, in modo da mantenere l'ordine delle fasi di lavoro.

1. Fase di analisi e ricerca:

La prima fase è stata quella di *analisi del project work*. Non era la prima volta che mi cimentavo nello sviluppo di algoritmi di machine learning, però non avevo mai affrontato il problema in ambito accademico.

Ho dunque ricercato sul web (in maniera particolare nella famosa piattaforma **Kaggle.com**) lo sviluppo di problemi simili, come ad esempio problemi di Natural Language Processing applicati a vari contesti, con lo scopo di apprendere nuove tecniche da poter considerare per il Project Work.

2. Sviluppo del progetto:

Una volta finito il penultimo esame, ovvero nel mese di Giugno 2026, ho iniziato lo sviluppo del progetto, suddiviso in *tre grandi macro-sezioni*, a seguire colorate in verde. In rosso invece i link ai relativi Jupyter Notebook.

Generazione Dataset, tempo di realizzazione: circa 2 settimane -

01_Generazione_Dataset_Elementare.ipynb e

02_Generazione_Dataset_Avanzato_Con_Gemma

Questa fase dava l'illusione di essere stata la più rapida da portare a termine, ma ho dovuto riprenderla a metà lavoro a seguito di un primo sviluppo della pipeline di Machine Learning in quanto i risultati non erano soddisfacenti.

I ticket, inizialmente generati **pseudo-casualmente** partendo da un *json* con al suo interno diverse *forme lessicali base*, contenevano delle **frasi spesso ripetitive, sconnesse e/o senza senso**. Ho ipotizzato dei possibili problemi causati dalla natura poco dinamica dell'algoritmo.

Link al file: [ticket_pseudorandom.csv](#)

Per curiosità stavo spulciando la fattura, ho scoperto che stornare la fattura con importo errato, e questo è un problema!	Amministrazione	Media
Salve, sono Rocco Colimprain. Ho scoperto che sollecitare il giustificativo di spesa:mancante della firma digitale	Amministrazione	Media
Gentili colleghi,vi ricontatto per sapere se ci sono novità in merito alla richiesta di monitorare un leak di memoria sulla macchina di produzione. Rimango a disposizione. Buon lavoro.	Tecnico	Alta
Riferimento le condizioni contrattuali. Potete verificare? è collegato a da inoltrare al commercialista. grazie anche da parte del capo.	Amministrazione	Alta
ciao, bisogna richiedere lo storno per il servizio cloud entro oggi se riusciamo... fatemi sapere, grazie	Commerciale	Bassa
scusate ma qui non funziona più nulla!!! dobbiamo assolutamente sollecitare il sollecito di pagamento o si blocca tutto il reparto!! rispondetemi asap	Amministrazione	Media
Ciao, per favore dovremmo rinnovare la proposta di partnership con opzione multilicenza il prima possibile.Fammi sapere se ci sono problemi. Grazie!	Commerciale	Bassa
Potete riscontrare i permessi di accesso in fase di caricamento? Grazie, saluti.	Tecnico	Bassa
Ciao, per favore dovremmo disdire il listino prezzi aggiornato per la nuova filiale il prima possibile.Fammi sapere se ci sono problemi. Grazie!	Commerciale	Media
Salve, si richiede di verificare il rimborso per duplicazione interna. Restiamo in attesa di un riscontro urgente.	Amministrazione	Alta
Salve, sono Arianna Dinon Fanel. Ho scoperto che sollecitare la fattura:per il pacchetto Enterprise	Amministrazione	Media
Salve, si richiede di approvare il listino prezzi aggiornato dopo l'ultimo aggiornamento. Restiamo in attesa di un riscontro urgente.	Tecnico	Media

Figura 1: Ticket sintetici creati con un algoritmo di generazione elementare

A questo punto, ho effettivamente iniziato a lavorare alla pipeline ML, e **le criticità ipotizzate si sono confermate**: il dataset portava gli algoritmi in *overfitting*: sebbene il classification report riportasse dei risultati ottimi (*vedi figura 6*), il modello si è rivelato **fragile nella generalizzazione**.

Ho dunque provato la **generazione con LLM commerciali** come *Gemini* e *ChatGPT*, ma il risultato era troppo semplicistico e soprattutto poco accademico.

Ho optato per qualcosa di più professionale, ovvero un'architettura che generasse dataset artificiali utilizzando un *LLM in locale*: **Gemma4**, nello specifico.

Questo processo ha richiesto un paio di giorni per essere perfezionato a causa della sua **natura più complessa** (anche dal punto di vista del livello hardware) e dei **trial and error** necessari fino al raggiungimento di un perfetto equilibrio per la generazione dei dati: è sì stato utilizzato un LLM, ma che ha lavorato in locale e alle mie condizioni. Non volevo che venisse generato un dataset per intero, bensì **un ticket per volta**.

La prima sfida (i primi 2-3 giorni) è stato il fatto che il **modello base** di Gemma4 creasse dei ticket **veramente realistici**, ma al tempo stesso **troppo complessi per gli algoritmi di ML**, senza considerare i **tempi di elaborazione troppo lunghi**; il tempo di generazione medio era di

circa 2 ticket al minuto, il che significava circa 120 ticket l'ora e di conseguenza circa 4 ore per la generazione di un dataset con 500 ticket.

Link al file: [ticket_gemma4_balanced.csv](#).

Vorrei segnalare che non riesco a accedere alla sezione pagamenti online. Ho provato due volte e mi dà sempre un messaggio di errore generico. Potreste darmi una mano, per favore? Grazie mille!	Amministrazione	Alta
Buongiorno! Eccomi di nuovo. Ho riscontrato un errore nell'autenticazione quando tento di accedere alla sezione relativa all'aggiornamento dei miei dati anagrafici. Potreste verificare la procedura o se ci sono aggiornamenti necessari sul mio account, per favore?	Amministrazione	Bassa
Ho necessità di segnalare un problema con l'integrazione dati. Ho provato a riconfigurare i parametri di connessione manualmente, ma il sistema continua a restituire errori di autenticazione 500. Potrei avere bisogno di una verifica sui permessi lato server o su eventuali dipendenze mancanti per completare la sincronizzazione.	Tecnico	Media
Ho già provato a riavviare il computer e ho verificato le credenziali. Non riesco ad accedere all'area clienti del CRM, mostrando un messaggio di errore 403. Potreste indicarmi i passaggi corretti o se è necessario un reset dei permessi utente? Grazie.	Tecnico	Media
Creo questo ticket perché non riesco a trovare il modulo di richiesta spese. Ho provato a cercare ovunque ma nulla appare. Questo è un casino e mi serve subito, grazie.	Amministrazione	Bassa
Dopo l'ultimo aggiornamento, volevo sapere se posso ancora approfittare dello sconto che mi avevate promesso. Ho appena visto i prezzi sul sito e non è chiaro come applicarlo al mio account. Grazie mille!	Commerciale	Bassa
Scusate il disturbo, devo capire perché quando guardo i prezzi per l'espansione mi esce un totale sbagliato. Ho provato a rifare il tutto due volte ma il sistema continua a sbagliare la somma finale. Potete darmi una mano con questa cosa? Grazie mille.	Commerciale	Media
Vi scrivo perché non riesco ad accedere al database clienti da stamattina. Ho provato a riavviare il portatile ma il messaggio di errore persiste ed è bloccato sull'autenticazione utente. Potreste verificare se ci sono problemi con le credenziali o con l'integrazione del sistema?	Tecnico	Alta
Vi scrivo perché non mi entra più nel portale aziendale. Ho provato a resettare la password ma continua a dare errore. Potete darmi una mano, per favore?	Tecnico	Alta

Figura 2: Ticket sintetici creati con l'algoritmo di generazione avanzato utilizzando Gemma4

Un risultato **più bilanciato e sostenibile** (un output di qualità leggermente inferiore, ma con tempi di elaborazione più che dimezzati) lo devo ad un *modello di Gemma più leggero*, ovvero **Gemma4:e2b**.

Il tempo di elaborazione è sceso a circa 5 ticket ogni minuto, dunque meno di 2 ore per la creazione di un dataset di 500 ticket. Questo ha permesso di effettuare molte più prove e di **modificare più volte la struttura del prompt base**, rendendolo in grado di reagire sempre in modi diversi ai parametri che lo generano. Link al file: [ticket_gemma4e2b.csv](#).

Ho bisogno di accesso immediato alla mia licenza software commerciale. Non riesco ad avviare le operazioni perché l'accesso è bloccato. Per favore risolvete subito questo problema.	Commerciale	Alta
Impossibile procedere con il lavoro a causa di un malfunzionamento immediato della licenza commerciale. Sono un nuovo utente e ho bisogno di attivare la chiave di accesso urgentemente per iniziare l'uso del servizio.	Commerciale	Alta
Ho un blocco totale nella gestione delle fatture e non riesco ad accedere al sistema. Questo è un problema di amministrazione molto urgente e devo risolvere subito.	Amministrazione	Alta
Ho un problema con l'aggiornamento delle licenze attuali e non riesco a procedere. Ho bisogno di una risoluzione urgente per evitare interruzioni operative.	Commerciale	Bassa
Ho notato un errore nella fattura di questo mese. Vorrei sapere come procedere per la correzione.	Commerciale	Bassa
Ho un problema di connessione VPN che blocca l'accesso ai sistemi critici. Ho già tentato il riavvio del client e la verifica delle credenziali, ma il problema persiste. Richiedo assistenza immediata per ripristinare la connettività.	Tecnico	Alta
Ho un'urgenza assoluta: Non riesco ad inserire i dati delle nuove fatture nel sistema. Il processo di input è bloccato e non so come procedere per la chiusura del mese.	Amministrazione	Alta
Ho avuto difficoltà a configurare la VLAN richiesta sulla switch. Non riesco ad applicare le policy di sicurezza come previsto. Mi serve aiuto per capire come procedere con il setup corretto.	Tecnico	Bassa
Ho riscontrato un blocco durante la fase di configurazione iniziale del software commerciale. Non riesco ad accedere alle funzionalità essenziali per il primo utilizzo. Richiedo assistenza urgente per risolvere questa situazione.	Commerciale	Alta
Per il futuro sarebbe utile capire se posso avere assistenza sulla corretta configurazione del sistema di gestione dei dati amministrativi. Ho riscontrato alcune difficoltà nell'inserimento delle voci e nella riconciliazione dei report mensili. Vorrei capire come procedere per risolvere questi problemi senza blocchi operativi.	Amministrazione	Bassa
Non riesco a attivare le mie licenze per il nuovo progetto. Mi aspettavo una soluzione immediata dato che sono un utente pigro e ho bisogno di una risposta media.	Commerciale	Media
Vi scrivo perché non riesco ad accedere al sistema di amministrazione per generare le fatture. Sono un nuovo utente e ho bisogno di completare questa operazione immediatamente.	Amministrazione	Alta
Non riesco a configurare il router. La procedura è troppo complicata e non capisco le istruzioni. Mi serve aiuto per la connessione di base, non voglio perdere tempo.	Tecnico	Bassa

Figura 3: Ticket sintetici creati con l'algoritmo di generazione avanzato utilizzando Gemma4:e2B

NOTA: il tempo di elaborazione può variare da dispositivo a dispositivo. A seguire uno screenshot delle caratteristiche hardware della macchina in cui è stata effettuata questa parte di progetto:

Processore	12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)
RAM installata	16,0 GB (15,6 GB utilizzabile)
Scheda grafica	NVIDIA GeForce RTX 4050 Laptop GPU (6 GB) Intel(R) UHD Graphics (128 MB)

Figura 4: Caratteristiche Hardware del computer responsabile della generazione

In alternativa, un'ottima soluzione per effettuare lo stesso esperimento potrebbe essere utilizzare piattaforme come [Google Colab](#), molto utilizzate proprio in contesti di Machine Learning grazie all'hardware che mette a disposizione in cloud.

Pipeline Machine Learning, tempo di realizzazione: circa 3 settimane -

03 Pipeline Machine Learning.ipynb

Questa è stata la fase dove mi sono soffermato maggiormente e anche **la più sfidante**.

Nonostante, come detto in precedenza, avessi già delle idee sul tipo di algoritmi da utilizzare, la sfida principale è stata quella di riuscire ad avere **un risultato ben bilanciato con i miei dati**.

Questo perché non tutti i dataset generati riuscivano a dare dei risultati soddisfacenti. È possibile consultare una **migliore analisi** dei risultati all'interno della **fase 6 del notebook**.

A seguire è possibile consultare i **classification report più interessanti** per l'evoluzione delle fasi di lavoro svolte, iniziando con una tabella che contiene l'insieme dei migliori risultati dei vari dataset.

Dataset	Accuracy Categoria	Accuracy Priorità
pseudorandom	0.89 con LinearSVC	0.86 RandomForest
gemma4_unbalanced	0.73 con LinearSVC	0.54 con Approccio Gerarchico
gemma4_balanced	0.82 con LinearSVC	0.53 con Approccio Gerarchico
gemma4e2b	0.84 con Random Forest	0.73 con Approccio Gerarchico
unione_di_tutti	0.93 con LinearSVC	0.80 con LinearSVC

Figura 5: Analisi dei migliori risultati degli algoritmi in base al dataset

Iniziamo con i risultati del **dataset realizzato pseudo randomicamente**. Come accennato prima, risultati eccellenti, ho sviluppato allora uno script che mi permettesse di **analizzare un ticket in input** (prototipo della futura dashboard finale), ma reagiva piuttosto male. Come previsto, utilizzando le parole nello stessa forma utilizzata durante la creazione, l'algoritmo classificava quasi sempre correttamente i ticket. **Cambiando però il contesto, perdeva le sue capacità di generalizzare**.

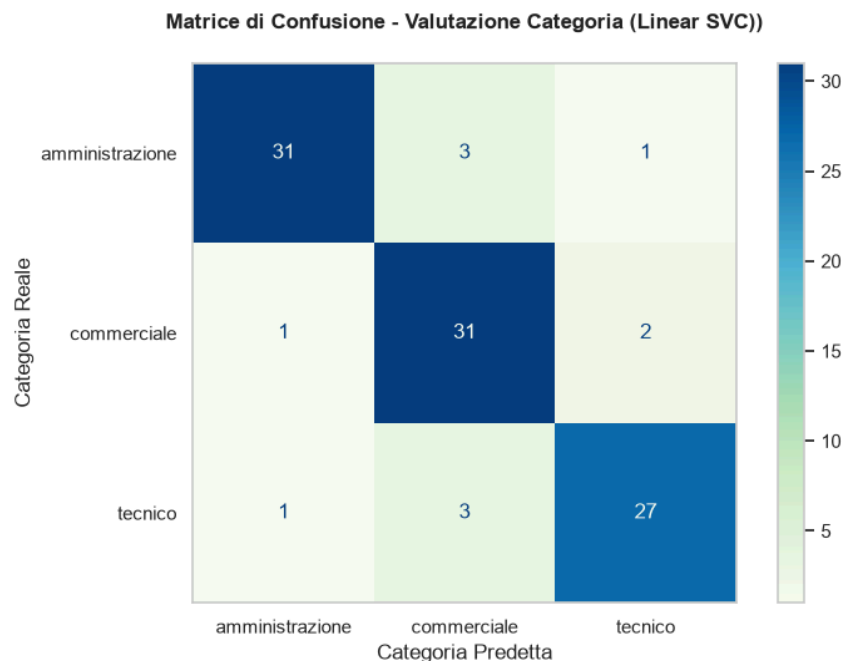


Figura 6.a: Matrice di confusione - classificazione categoria dataset pseudorandom

```

--- CLASSIFICATION REPORT - VALUTAZIONE CATEGORIA (Linear SVC) ---
Accuracy Totale: 0.8900

Report di Classificazione (comprende F1-Macro):

```

	precision	recall	f1-score	support
amministrazione	0.94	0.89	0.91	35
commerciale	0.84	0.91	0.87	34
tecnico	0.90	0.87	0.89	31
accuracy			0.89	100
macro avg	0.89	0.89	0.89	100
weighted avg	0.89	0.89	0.89	100

Figura 6.b Classification report - classificazione categoria dataset pseudorandom

Nutrido grandi speranze per i dataset generati dalla **versione standard di Gemma4**, ma invece la **matrice di confusione** fa da bandiera ad uno **schema problematico**.

Con un **accuracy di 0.51**, possiamo considerare questi risultati niente di più rispetto al **lancio di una moneta**.

I maggiori imputati a questi problemi sono:

- Frasi troppo complesse e ricche di vocaboli per algoritmi di machine learning semplici
- Pochi dati in input (500 ticket totali)
- Dataset creato volontariamente sbilanciato per il numero delle priorità per simulare la vita reale (statisticamente è più facile che una segnalazione sia con priorità bassa che alta). Da qui viene il nome del dataset.

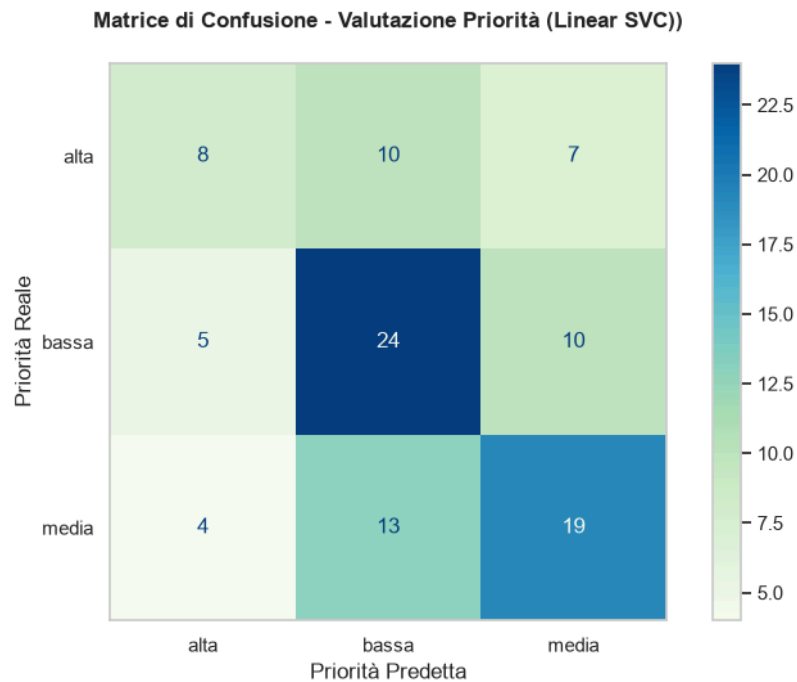


Figura 7.a: Matrice di confusione - valutazione priorità dataset gemma4_unbalanced

--- CLASSIFICATION REPORT - VALUTAZIONE PRIORITÀ (Linear SVC) ---
 Accuracy Totale: 0.5100

Report di Classificazione (comprende F1-Macro):

	precision	recall	f1-score	support
alta	0.47	0.32	0.38	25
bassa	0.51	0.62	0.56	39
media	0.53	0.53	0.53	36
accuracy			0.51	100
macro avg	0.50	0.49	0.49	100
weighted avg	0.51	0.51	0.50	100

Figura 7.b: Classification report - valutazione priorità dataset gemma4_unbalanced

Ho provato a risolvere quantomeno il terzo problema, generando altri ticket con la stessa procedura bilanciando il numero di ticket per priorità ma nulla, i risultati sono **addirittura peggiorati**.

Con l'algoritmo Random Forest Classifier l'accuracy è addirittura **scesa a 0.39**. Qui davvero era un po' come se il modello lanciasse un **dado a 3 facce** per scegliere una tra le tre priorità.

Matrice di Confusione - Valutazione Priorità (Random Forest Classifier)

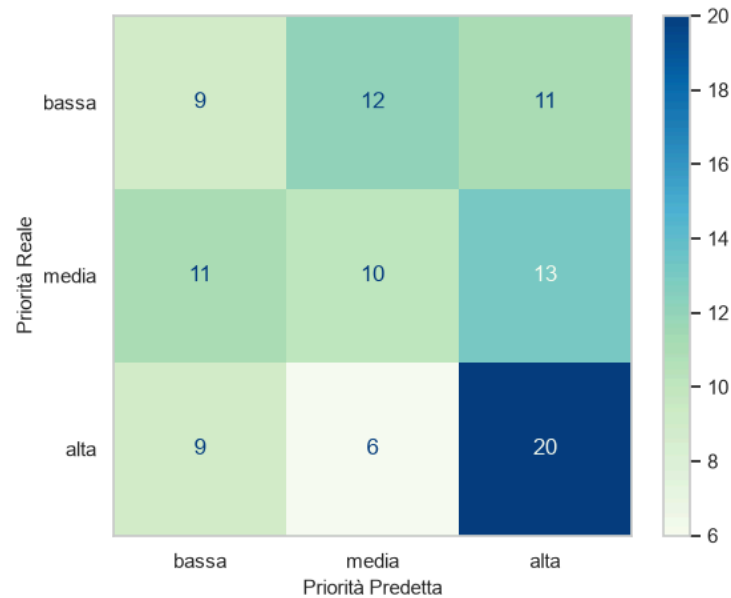


Figura 8.a: Matrice di confusione - valutazione priorità dataset gemma4_balanced

--- CLASSIFICATION REPORT - VALUTAZIONE PRIORITÀ (Random Forest Classifier) ---
Accuracy Totale: 0.3861

	precision	recall	f1-score	support
alta	0.45	0.57	0.51	35
bassa	0.31	0.28	0.30	32
media	0.36	0.29	0.32	34
accuracy			0.39	101
macro avg	0.37	0.38	0.37	101
weighted avg	0.38	0.39	0.38	101

Figura 8.b: Classification report - valutazione priorità dataset gemma4_balanced

Ho deciso di svolgere un **ultimo esperimento** prima di cercare altre soluzioni. Ho utilizzato lo stesso codice, ma sfruttando un modello di Gemma4 più leggero, la già citata **Gemma4:e2b**. La sua leggerezza è dovuta proprio dal **basso numero di parametri** e, di conseguenza, dal suo vocabolario **meno complesso**. Questa è stata la soluzione vincente.

In combinazione ad un'**operazione di stemming** durante il preprocessing e ad un **approccio gerarchico dell'analisi** (entrambi processi descritti nella sezione *descrizione dei principali aspetti progettuali*) i risultati sono migliorati e non di poco, come è possibile notare dalla matrice di confusione e come testimonia abilmente la dashboard di cui stiamo per fare la conoscenza.

Matrice di Confusione - Valutazione Priorità (Approccio gerarchico LinearSVC)

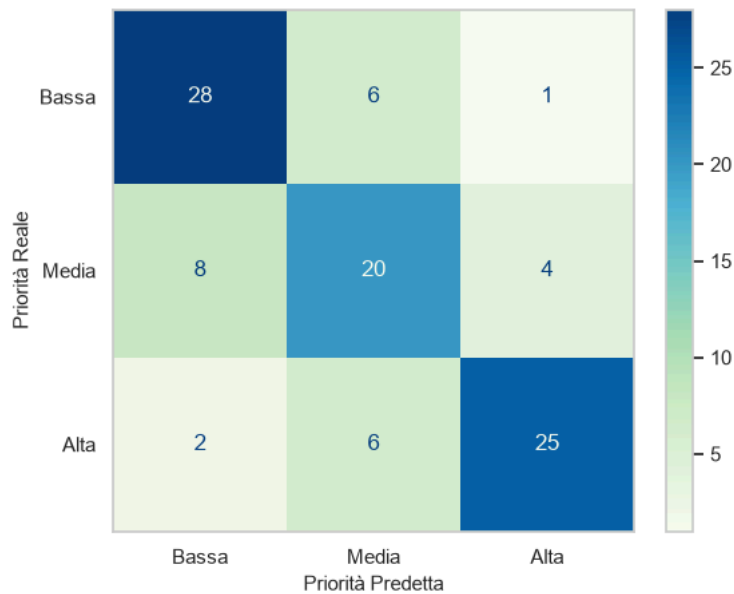


Figura 9.a: Matrice di confusione - valutazione priorità dataset gemma4e2b

--- CLASSIFICATION REPORT - VALUTAZIONE PRIORITÀ (Approccio gerarchico Random Forest) ---
Accuracy Complessiva sulla Priorità: 0.6500

	precision	recall	f1-score	support
Alta	0.76	0.85	0.80	33
Bassa	0.68	0.54	0.60	35
Media	0.51	0.56	0.54	32
accuracy			0.65	100
macro avg	0.65	0.65	0.65	100
weighted avg	0.65	0.65	0.65	100

Figura 9.b: Classification report - valutazione priorità dataset gemma4e2b

In conclusione, il **problema più grande** affrontato in questa fase è stato sicuramente riuscire a trovare dei risultati soddisfacenti per quanto riguarda la **valutazione delle priorità**, unico punto in cui gli algoritmi di machine learning utilizzati hanno mostrato maggiori debolezze.

Creazione di una web app per la dashboard, tempo di realizzazione: circa 2 giorni - Dashboard.py

Questa è stata infine la **sezione più leggera** da elaborare dell'intero progetto. Inizialmente avevo optato per una UI sviluppata utilizzando la libreria **tkinter** ma la sua resa grafica non **era soddisfacente**.

Dopo essermi documentato su delle soluzioni alternative, la scelta è ricaduta su **NiceGUI**, ovvero un framework per l'UI che viene visualizzato su interfaccia web.

Una volta creato il codice ed averlo testato in locale, ho impiegato circa un'ora dello sviluppo del progetto per effettuare un **deploy su Render** con lo scopo di rendere la web app di **facile accesso** senza necessità di eseguire alcun codice nella propria macchina.

Link alla web app: <https://project-work-vdxq.onrender.com/>.

NOTA: nel primo avvio potrebbe essere necessario aspettare qualche minuto che Render mandi in esecuzione lo script python.

Ticket:

Oggetto (Titolo)
Stampante inceppata

Corpo del Ticket (Descrizione del problema)
La stampante condivisa in corridoio segnala un errore sul toner. Quando passate potreste dargli un'occhiata?

ANALIZZA TICKET

Risultato analisi del ticket

Categoria prevista:
TECNICO

Priorità suggerita:
BASSA

Top 5 parole più influenti (categoria e priorità):

errore potreste sul
un errore la sul quando
potreste un

Figura 10: Interfaccia grafica della web app

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Risorse utilizzate (bibliografia, documentazioni, ecc...):

Lo sviluppo della pipeline e l'implementazione dei modelli hanno richiesto la consultazione di documentazioni e bibliografie degli strumenti adottati.

- [Documentazione Scikit-Learn](#), consultata in particolar modo per il modulo di estrazione delle feature testuali (TfidfVectorizer) e per il tuning degli algoritmi di classificazione.
- [Sito Speech and Language Processing \(3rd ed. draft\)](#), un riferimento accademico riguardo la storia del Language Processing.
- [Documentazione Ollama](#), utilizzata per l'installazione, configurazione e gestione via riga di comando dei modelli LLM ospitati in locale.
- [Documentazione NiceGUI](#), utilizzata per apprendere le funzionalità del framework durante lo sviluppo della dashboard.
- [Sito PyPi](#), per la documentazione delle varie librerie utilizzate nel progetto.
- [Sito Snowflake Fundamentals](#), piattaforma consultata per la definizione tecnica e i principi di funzionamento dell'algoritmo di apprendimento Random Forest.
- [Sito Wikipedia](#), consultato per definizioni formali inserite all'interno del Jupyter o di questo elaborato, come ad esempio il funzionamento teorico dell'algoritmo LinearSVC.
- [Sito Gemma4 Model Card](#), documentazione tecnica ufficiale fornita da Google per comprendere le specifiche e le potenzialità dei modelli Gemma4.

Banche dati utilizzate:

- Per la fase di generazione pseudo-casuale della prima versione del dataset, è stato necessario reperire una lista di nomi e cognomi per generare dei record verosimili. In particolare, ho trovato a seguito di una ricerca un [repository](#) all'interno di GitHub gestita dall'utente *Max1234-Ita*. Partendo da queste liste, ho generato dei csv contenenti cognomi e nomi italiani.

Strumenti utilizzati per lo sviluppo:

Gli strumenti principali utilizzati per lo sviluppo del progetto sono:

- **Ollama**, uno strumento open source che permette di scaricare, configurare ed eseguire **modelli linguistici** di grandi dimensioni (LLM) direttamente sul proprio computer. Come descritto nel notebook “[02_Generazione_Dataset_Avanzato_Con_Gemma.ipynb](#)”, per la creazione di un dataset avanzato si è scelto di **sfruttare la potenza degli LLM**, sviluppando una pipeline capace di **generare dei ticket** a partire da un prompt dinamico con parametri casuali.

Per il suo corretto utilizzo è necessario **installare il software** nel proprio sistema operativo. Link al download nel loro [sito ufficiale](#).

Analizzo adesso i modelli utilizzati:

- **Gemma4**: è una **famiglia di modelli di intelligenza artificiale** open-weight sviluppati da Google DeepMind, progettati per garantire un'elevata efficienza e un'eccellente capacità di ragionamento direttamente **in locale su computer**, smartphone e dispositivi edge.
- **Gemma4:e2b**: è uno dei modelli di intelligenza artificiale open-weight più compatti della famiglia **Gemma 4** di Google. La sigla "*E2B*" indica **2,3 miliardi di parametri effettivi**.
Questa architettura ha permesso di **aggirare i vincoli** legati al rate limiting e i costi di consumo tipici delle API basate su cloud (come ad esempio *Gemini*, anch'essa di Google), garantendo al contempo il **controllo totale sui parametri di generazione** (come ad esempio la formattazione nativa della risposta).
Per la sua installazione basta utilizzare il comando `ollama run gemma4:e2b`. Per la sua rimozione invece occorre utilizzare `ollama rm gemma4:e2b`.
- **Render**, una piattaforma cloud progettata per **semplificare il deploy e l'hosting di applicazioni web**. All'interno di questo project work è stato utilizzato per mettere in produzione la *dashboard interattiva*, collegandosi direttamente al repository GitHub per automatizzare il processo di pubblicazione. Grazie a questa infrastruttura, la dashboard è utilizzabile pubblicamente tramite un *qualsiasi comune browser web* al seguente [link](#).
- **Visual Studio Code**, IDE di riferimento, scelto per la sua versatilità e per la familiarità acquisita in ambito professionale. È stato utilizzato per la maggior parte del lavoro, dalla stesura del codice all'analisi dei dati in output. Tra le estensioni fondamentali utilizzate troviamo:
 - **Jupyter**, un'estensione di *Microsoft* che funge da supporto per Jupyter Notebook, per una programmazione interattiva ed elaborazione dati con supporto per IntelliSense, debug e altro ancora.
 - **CSV**, un'estensione di *ReprEng* che permette di visualizzare i file CSV in formato tabellare, che ha reso la visualizzazione dei dataset generati molto più chiara e comprensibile.
 - **LaTeX Workshop**, un'estensione di *James Yu* che migliora l'efficienza dell'impaginazione in LaTeX grazie alle funzioni di anteprima, compilazione, completamento automatico, colorazione e altro ancora.
- **Python**, **linguaggio di programmazione** utilizzato per la creazione del Project Work. All'interno del GitHub è presente un file *requirements.txt* con all'interno le **librerie da installare** utilizzando il comando `pip`.

Eccone però una breve descrizione con i relativi link alle loro descrizioni su [PyPI.org](#):

- **pandas**: Lettura, manipolazione e salvataggio dei dataset CSV in tutte le fasi del progetto.
- **scikit-learn**: Motore della pipeline di Machine Learning. Contiene gli strumenti per vettorizzare il testo (*TfidfVectorizer*), suddividere i dati (*train_test_split*), addestrare i modelli (*LinearSVC*, *RandomForestClassifier*) e calcolare le metriche di valutazione (*accuracy_score*, *classification_report*, *confusion_matrix*).
- **matplotlib**: Necessaria per la visualizzazione delle matrici di confusione e per le basi della data visualization.
- **seaborn**: Utilizzata per l'Analisi Esplorativa dei Dati (*EDA*) per creare grafici avanzati
- **ollama**: Il client ufficiale in Python per comunicare con il server Ollama locale ed utilizzare Gemma4:e2b.
- **pydantic**: Utilizzata nella generazione avanzata con Gemma 4:e2b per strutturare, tipizzare e forzare l'output del modello linguistico in un formato JSON rigoroso.
- **niceGUI**: NiceGUI è un framework per l'interfaccia utente basato su Python e di facile utilizzo, che viene visualizzato nel browser web. È possibile creare pulsanti, finestre di dialogo, Markdown, scene 3D, grafici e molto altro ancora.
- **GitHub**, utilizzato come piattaforma di **version control** e **repository centralizzato** per l'intero progetto, fungendo inoltre da ponte strategico per il *deploy della dashboard* su Render. Il suo impiego si è rivelato fondamentale non solo per l'archiviazione sicura del codice sorgente, ma anche per garantire un'ottimale **sincronizzazione del lavoro** tra i diversi dispositivi utilizzati durante le sessioni di sviluppo.
- **Gemini**, assistente basato su intelligenza artificiale sviluppato da Google. Utilizzato come **strumento di supporto** per velocizzare le ricerche e l'EDA dei dataset prima dello sviluppo degli algoritmi ad hoc. Utilizzato inoltre come supporto per la **revisione grammaticale** dell'elaborato.
- **Canva**, piattaforma online per la **creazione di grafiche**, utilizzata per le illustrazioni nella sezione “Campi di applicazione” dell'elaborato.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

Il Project Work pone l'obiettivo primario di **progettare, sviluppare e validare un prototipo software essenziale**, modulare e completamente riproducibile per l'**automatizzazione del triage di ticket**, seguendo le specifiche richieste dalla traccia PW18.

Descrizione della traccia:

“Un'azienda riceve ogni giorno decine di ticket via email o modulo web [...] Lo smistamento manuale rallenta i tempi di risposta e crea frustrazione. L'obiettivo del project work è realizzare un prototipo essenziale e riproducibile che [...] classifichi automaticamente ogni ticket nella categoria più pertinente e suggerisca una priorità operativa [...]”

L'elaborato riesce a **trasformare un problema operativo tipico** degli help desk (il collo di bottiglia generato dalla lettura e dallo smistamento manuale di comunicazioni non strutturate) in una **pipeline** di *Natural Language Processing* (NLP) efficiente che **automatizza le procedure**.

Nel dettaglio, il lavoro ha permesso di raggiungere i seguenti **macro-obiettivi**:

1. Smistamento mirato sulle aree di business aziendali:

Come richiesto dalle specifiche, il sistema **prende in input un testo grezzo** e restituisce in output una **classificazione automatica del ticket**:

- *Assegnazione della Categoria* -> smistamento verso il reparto competente (Amministrazione, Tecnico o Commerciale)

- *Suggerimento della Priorità* -> valutazione dell'urgenza per ottimizzare i tempi di gestione su ticket critici.

2. Generazione dei dati “ad hoc”:

Seguendo i vincoli di generazione “*senza informazioni personali*” e partendo da un “*piccolo dataset sintetico di ticket brevi (oggetto + descrizione)*”, l'elaborato ha raggiunto l'obiettivo di creare **dataset completamente artificiali ma semanticamente verosimili**.

Sfruttando la generazione avanzata tramite **modelli LLM installati in locale** (*Gemma 4:e2b*) e controllati tramite schemi *Pydantic*, sono stati generati **ticket strutturati** esattamente in Titolo e Corpo del messaggio, privi di qualsiasi dato sensibile ma **ricchi del lessico e delle sfumature** necessarie all'addestramento.

3. Semplicità del codice e chiarezza di flusso:

Rispondendo al requisito “*tecniche di ML di base mantenendo il focus su semplicità e chiarezza*”, l'architettura è stata sviluppata utilizzando **semplici algoritmi di classificazione supervisionati**, ma comunque altamente performanti per il NLP: **LinearSVC** e **RandomForestClassifier**, utilizzando **TfidfVectorizer** per la vettorizzazione.

Il flusso di lavoro risulta dunque veramente semplice ed intuitivo.



Figura 11: Flusso di lavoro del Project Work

4. Verifica dell'affidabilità:

Per verificare che il modello fosse “*ragionevolmente affidabile*”, ogni esperimento condotto nei **Jupyter Notebook** è stato accompagnato da **Classification Report** (utilizzo di Accuracy, F1-score e Matrici di Confusione) che permettono di validare scientificamente il prototipo.

```

--- CLASSIFICATION REPORT - VALUTAZIONE CATEGORIA (Linear SVC) ---
Accuracy Totale: 0.8300

Report di Classificazione (comprende F1-Macro):
              precision    recall  f1-score   support

 amministrazione    0.85     0.91     0.88        32
  commerciale      0.78     0.81     0.79        36
    tecnico        0.86     0.78     0.82        32

 accuracy              0.83              100
 macro avg           0.83     0.83     0.83        100
 weighted avg        0.83     0.83     0.83        100
  
```

Figura 12: Esempio di classification report

5. Riproducibilità e design minimale per un'esecuzione rapida:

La caratteristica “*un prototipo essenziale e riproducibile*” e che “*permetta di eseguire e comprendere il prototipo in pochi minuti*” viene soddisfatta dalla stesura dei **Jupyter Notebook** ampiamente commentati, eseguibili in blocco su **qualsiasi computer in locale o in cloud** (ad esempio Google Colab), e dal deploy di una [web app per la dashboard](#), facilmente **fruibile tramite qualsiasi browser web** senza necessità di alcuna installazione o configurazione tecnica.

NOTA: l'unico elemento che potrebbe creare rallentamenti riguardo la riproducibilità è il notebook per la creazione del dataset usando Gemma, a causa della sua maggiore complessità sia progettuale che di elaborazione. Ho deciso di non rimuovere la prima versione del codice anche

per questo motivo. All'interno della cartella dataset sono disponibili diversi .csv generati pronti all'uso. Link alla cartella: <https://github.com/gabrieleStagnitta/Project-Work/tree/main/dataset>

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

Contesto teorico:

Dal punto di vista scientifico, l'elaborato si inserisce nel dominio del **Natural Language Processing (NLP)** e della **Classificazione Testuale Supervisionata**.

- Storicamente parlando, i **primi sistemi di smistamento automatico** dei testi si basavano su approcci statici con **regole basate su parole chiave**. Sistemi molto semplici da implementare, però al tempo stesso **molto fragili alla complessità del linguaggio** naturale. Il contesto teorico di questo elaborato si fonda proprio sul **superamento di tali limiti** tramite il **Machine Learning statistico**, in cui gli algoritmi comprendono in maniera supervisionata i **pattern semantici e le relazioni con il gruppo di appartenenza**.
- La letteratura accademica moderna è dominata da architetture Deep Learning, Transformer, ecc... come ad esempio **BERT** (Bidirectional Encoder Representations from Transformers), oppure **fastText**, per task di classificazione di testi brevi, **algoritmi di Machine Learning tradizionali** rappresentano comunque un'ottima soluzione. In maniera particolare, gli algoritmi utilizzati sono il **LinearSVC** e **Random Forest Classifier**, con il sostegno di **TfidfVectorizer** per l'estrazione delle feature.
- Un ulteriore pilastro teorico del progetto è l'**utilizzo di LLM** (nel caso specifico Gemma4:e2b) non come strumento di generazione dataset per intero, bensì come **generazione atomica dei ticket** seguendo prompt generati casualmente **in runtime**. Ritengo questo elemento **un grosso punto di forza** dell'intero progetto in quanto è stato essenziale per rispondere ad un problema classico del Machine Learning, ovvero **l'assenza di dati etichettati di alta qualità in contesti creati da zero**, cosa che in fase di progettazione stava inizialmente creando non pochi problemi.

Contesto applicativo:

Dal punto di vista applicativo, il progetto si inserisce nel **settore del CRM** e dei **sistemi di Smart Ticketing**.

- Nelle aziende di media-larga dimensione, la ricezione quotidiana di **comunicazioni non strutturate** tramite diversi canali può essere un problema. Lo smistamento manuale genera **colli di bottiglia** che rallentano il lavoro degli operatori e lo espongono a potenziali errori di assegnazione. Inoltre implica del lavoro umano ripetitivo e altamente alienante.
- In uno scenario reale, l'elaborato avrebbe il compito di agire come da **“primo filtro”**, che si fa carico proprio del processo ripetitivo descritto nel punto precedente e lo trasforma in un **flusso di lavoro misurabile e tracciabile**, lasciando all'operatore il solo compito di **supervisione e risoluzione effettiva del problema**.

Descrizione dei principali aspetti progettuali

(Sviluppare l'elaborato richiesto dalla traccia prescelta):

La realizzazione operativa del Project Work è stata concepita seguendo un approccio di tipo **end-to-end**, strutturando il flusso di lavoro in diversi moduli **indipendenti e interconnessi** tra loro.

Proprio come Virgilio accompagna Dante nel suo complesso viaggio attraverso i vari gironi dell'Inferno, l'architettura del sistema si pone come “guida” del dataset **dalla sua generazione**

artificiale, passando poi per le **fasi di pulizia e vettorizzazione NLP**, fino all'**addestramento dei modelli predittivi** e alla restituzione del risultato in un'**interfaccia utente web** reattiva e accessibile.

All'interno di questo documento saranno presenti solo **le parti di codice che vale la pena analizzare**.

Per garantire la massima **trasparenza, riproducibilità e verificabilità del codice** condivido il link della **repository GitHub principale**.

Inserisco anche il link utile per l'**accesso alla web app** e alla **repository GitHub** utile al suo mantenimento.

[Link repository principale](#)

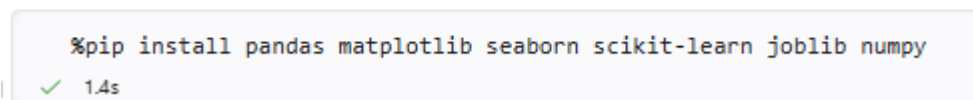
[Link repository per la dashboard](#)

[Link web app dashboard](#)

All'interno della repository principale è presente un file [README.md](#) che illustra nel dettaglio la struttura e l'organizzazione delle cartelle e dei file.

I vari file Jupyter Notebook contengono già al loro interno una dettagliata descrizione delle varie fasi trattate al loro interno. Inoltre è sempre presente una cella di codice con all'interno il comando per l'installazione delle librerie utilizzate nel progetto, come ad esempio:

Installazione di librerie utilizzate nel progetto:



```
%pip install pandas matplotlib seaborn scikit-learn joblib numpy
```

✓ 1.4s

Figura 13: Esempio cella di comando per installazione librerie

In ogni caso, è consigliata la creazione di un virtual environment e l'installazione delle librerie tramite il file [requirements.txt](#).

Generazione Dataset elementare - 01 Generazione Dataset Elementare.ipynb

In questo **Jupyter Notebook** è possibile consultare una soluzione elementare per la **realizzazione del primo obiettivo** del Project Work. Gli obiettivi specifici raggiunti in questa sezione sono:

- *Creazione di un piccolo dataset sintetico (200–500 ticket testuali) con etichetta di categoria e priorità.*
 - *Sintesi di ticket brevi (titolo + descrizione) con lessico tipico per Amministrazione, Tecnico, Commerciale.*
 - *Etichette di priorità semplici basate su parole chiave (es. 'errore', 'fattura', 'bloccante').*
 - *Esportazione CSV con colonne: id, title, body, category, priority.*

Come prima cosa, è stato generato un file .json con all'interno un vocabolario da utilizzare per realizzare i ticket. Il JSON contiene una divisione per 3 tipologie di ticket, come da consegna:

- Amministrazione
- Tecnico
- Commerciale

Ogni categoria contiene a sua volta 3 liste, ovvero:

- Verbi
- Sostantivi
- Dettagli dei ticket (frasi o contesti specifici)

Link al file: <https://github.com/gabrieleStagnitta/Project-Work/blob/main/input/lessico.json>

```
1 {
2   "Amministrazione": {
3     "verbi": [
4       "sollecitare", "verificare", "restituire", "rettificare", "inviare", "ricevere", "assistere",
5       "contabilizzare", "approvare", "stornare", "sollecitare", "archiviare", "sospendere"
6     ],
7     "sostantivi": [
8       "la fattura", "il pagamento", "la nota di credito", "lo storno", "il rimborso",
9       "l'estratto conto", "la distinta di bonifico", "il sollecito di pagamento", "il giustificativo di spesa", "la nota spese"
10    ],
11    "dettagli": [
12      "del mese scorso", "per il servizio cloud", "con importo errato", "non ancora pervenuto",
13      "relativo al trimestre in corso", "mancante della firma digitale", "da inoltrare al commercialista", "per duplicazione interna"
14    ]
15  },
16  "Tecnico": {
17    "verbi": [
18      "riscontrare", "segnalare", "risolvere", "riavviare", "ottimizzare",
19      "configurare", "ripristinare", "monitorare", "aggiornare", "testare", "isolare"
20    ],
21    "sostantivi": [
22      "un errore 500", "un bug bloccante", "il crash dell'app", "il server offline", "il database lento",
23      "un leak di memoria", "il certificato SSL scaduto", "il backup notturno", "i permessi di accesso", "il picco di traffico anomalo"
24    ],
25    "dettagli": [
26      "dopo l'ultimo aggiornamento", "sulla macchina di produzione", "durante il login", "in fase di caricamento",
27      "sull'ambiente di staging", "che impatta tutti gli utenti", "da mobile e da desktop", "subito dopo il deploy di stamattina"
28    ]
29  },
30  "Commerciale": {
31    "verbi": [
32      "richiedere", "valutare", "attivare", "rinnovare", "modificare",
33      "negoziare", "disdire", "proporre", "estendere", "sbloccare"
34    ],
35  }
```

Figura 14: Parte del contenuto del file lessico.json

Lo spezzone di codice a seguire mostra la logica che sta alla base della **generazione delle frasi** che popoleranno i ticket.

All'interno della variabile *lessico* è stato precedentemente conservato il **contenuto del JSON**. All'interno del ciclo for, avviene invece la generazione di un singolo ticket: vengono estratti in maniera pseudocasuale tutti i componenti che faranno parte del ticket finale. Prima la **categoria**, poi casualmente si sceglie se inserire o meno del **rumore artificiale**, aggiungendo parole prese da altre categorie (per evitare overfitting). Si scelgono i **verbi**, **sostantivi**, e i **dettagli** da utilizzare nel ticket, tutti parametri da inserire a dei formati preimpostati, che presentano delle frasi con una determinata struttura lessicale.

```

pesi_oggetti = [0.18,0.22,0.10,0.12,0.05,0.10,0.10,0.05,0.08]
pesi_formati = [0.12,0.15,0.12,0.08,0.05,0.08,0.08,0.08,0.02,0.02,0.02,0.02,0.07,0.07]

utenti_estratti = random.choices(pool_utenti, k=num_ticket)
dataset = []

for i in range(num_ticket):
    utente = utenti_estratti[i]
    cat_reale = random.choice(categorie)

    # C'è una probabilità del 15% che il ticket usi parole incrociate da altre categorie
    cat_verbo = (
        cat_reale if random.random() > 0.15 else random.choice(categorie)
    )
    cat_sost = (
        cat_reale if random.random() > 0.15 else random.choice(categorie)
    )
    cat_dett = (
        cat_reale if random.random() > 0.15 else random.choice(categorie)
    )

    v = random.choice(lessico[cat_verbo]["verbi"])
    s = random.choice(lessico[cat_sost]["sostantivi"])
    d = random.choice(lessico[cat_dett]["dettagli"])

    oggetti = [
        f"{v.capitalize()} {s}",
        f"URGENTE: {v.capitalize()} {s}",
        f"Richiesta supporto - categoria: {cat_reale}",
        f"Segnalazione {s}",
        f"AIUTO - problema con {s}",
        f"Ticket {v.capitalize()} {s}",
        f"Richiesta info {s} {d}",
        f"{s} - {d}",
        f"problema urgente {s} non va",
    ]
    title = random.choices(oggetti, weights=pesi_oggetti)[0]

    formati = [
        f"Buongiorno, con la presente si richiede cortesemente di {v} {s} {d}. Restiamo in attesa di un vostro riscontro urgente. Cordiali saluti.",
        f"Ciao, per favore dovremmo {v} {s} {d} il prima possibile. Fammi sapere se ci sono problemi. Grazie!",
        f"Salve, si richiede di {v} {s} {d}. Restiamo in attesa di un riscontro urgente.",
    ]

```

Figura 15: Implementazione della generazione procedurale con pesi distribuiti e iniezione di rumore semantico

Infine, si attribuisce un **livello di priorità**, che dipende dalle parole sorteggiate al suo interno. Il tutto alla fine viene **conservato in un file .csv di output**.

```

testo_completo = f"{title} {body}".lower()

conteggio_alta = sum(1 for kw in keywords_alta if kw in testo_completo)
conteggio_media = sum(1 for kw in keywords_media if kw in testo_completo)

if conteggio_alta >= 2 and conteggio_media <= 1:
    priority = "Alta"
elif conteggio_media >= 2 and conteggio_alta <= 1:
    priority = "Media"
elif conteggio_alta > conteggio_media:
    priority = "Alta"
elif conteggio_media > 0:
    priority = "Media"
else:
    priority = "Bassa"

dataset.append(
    {
        "id": f"TKT-{i+1:04d}",
        "user_name": utente["user_name"],
        "user_email": utente["user_email"],
        "title": title,
        "body": body,
        "category": cat_reale,
        "priority": priority,
    }
)

# Scrittura finale ottimizzata
df = pd.DataFrame(dataset)
df.to_csv(nome_file_output, index=False, encoding="utf-8-sig")

print(
    f"Successo! Creato il file {nome_file_output} con {num_ticket} righe totali."
)

```

Figura 16: Attribuzione della priorità e salvataggio su file output

Questo script basato su regole si comporta già abbastanza bene, ma non ero ancora soddisfatto dell'output finale. La **pseudo randomizzazione** che contraddistingue lo script rende i ticket innaturali e, nella maggior parte dei casi, **molto ripetitivi**. Per superare tale limite, ho deciso dunque di sfruttare la potenza dei modelli generativi, che verrà analizzato meglio nel prossimo Jupyter Notebook. Al seguente [link](#) è possibile visualizzare un dataset generato con l'architettura appena descritta.

problema urgente la nota di credito non va	Salve, si richiede di contabilizzare la nota di credito del mese scorso. Restiamo in attesa di un riscontro urgente.	Amministrazione	Media
URGENTE: Proporre lo sconto commerciale	Gentili colleghi,vi ricontatto per sapere se ci sono novità in merito alla richiesta di proporre lo sconto commerciale per un potenziale cliente. Rimango a disposizione. Buon lavoro.	Commerciale	Alta
la distinta di bonifico - da inoltrare al commercialista	Buongiorno,con la presente si richiede cortesemente di stornare la distinta di bonifico da inoltrare al commercialista. Restiamo in attesa di un vostro riscontro urgente.Cordiali saluti.	Amministrazione	Media
URGENTE: Monitorare il backup notturno	Ciao, per favore dovremmo monitorare il backup notturno subito dopo il deploy di stamattina il PRIMA possibile.Fammi sapere se ci sono problemi. Grazie!	Tecnico	Alta
URGENTE: Approvare il giustificativo di spesa	ciao, bisogna approvare il giustificativo di spesa del mese scorso entro oggi se riusciamo... fatemi sapere, grazie	Amministrazione	Alta
Segnalazione il rimborso	Potete restituire il rimborso con importo errato? Grazie, saluti.	Amministrazione	Media
Richiesta inof il crash dell'app da mobile e da desktop	Buongiorno,con la presente si richiede cortesemente di stornare il crash dell'app da mobile e da desktop. Restiamo in attesa di un vostro riscontro urgente.Cordiali saluti.	Amministrazione	Alta
problema urgente lo sconto commerciale non va	[NOTIFICA AUTOMATICA]: Rilevata necessità di valutare lo sconto commerciale per un potenziale cliente. Si prega di non rispondere a questa email.	Commerciale	Media
Riavviare il pico di traffico anomalo	Buongiorno,con la presente si richiede cortesemente di riavviare il picco di traffico anomalo per la nuova filiale. Restiamo in attesa di un vostro riscontro urgente.Cordiali saluti.	Tecnico	Alta
il giustificativo di spesa - valido solo fino a fine mese	Ciao, per favore dovremmo sollecitare il giustificativo di spesa valido solo fino a fine mese il prima possibile.Fammi sapere se ci sono problemi. Grazie!	Amministrazione	Media
URGENTE: SOLLECITARE la distinta di bonifico	Riferimento la distinta di bonifico. Potete verificare? è collegato a per il servizio cloud. grazie anche da parte del capo.	Amministrazione	Media
Segnalazione le condizioni contrattuali	Ciao, per favore dovremmo stornare le condizioni contrattuali valido solo fino a fine mese il prima possibile.Fammi sapere se ci sono problemi. Grazie!	Commerciale	Media
Aggiornare i permessi di accesso	Gentili colleghi,vi ricontatto per sapere se ci sono novità in merito alla richiesta di aggiornare i permessi di accesso che impatta tutti gli utenti. Rimango a disposizione. Buon lavoro.	Tecnico	Bassa
URGENTE: Sospendere la fattura	ciao, bisogna sospendere la fattura per duplicazione interna entro oggi se riusciamo... fatemi sapere, grazie	Amministrazione	Alta
Testare un leak di memoria	ho un problema, quando posso chiamarvi?	Tecnico	Media
Richiesta supporto - categoria: Tecnico	[NOTIFICA AUTOMATICA]: Rilevata necessità di isolare i permessi di accesso durante il login. Si prega di non rispondere a questa email.	Tecnico	Bassa

Figura 17: Estratto del dataset generato

Generazione Dataset con Gemma4:e2b - 02_Generazione Dataset Avanzato Con Gemma

Se l’approccio elementare ha permesso di **immaginare la struttura** che avrebbe dovuto impiegare il dataset, è solo con l’integrazione dei LLM che si è riusciti a conferire ai ticket quella **complessità, imprevedibilità e sfumature lessicali** che caratterizzano le **reali comunicazioni** tra esseri umani. L'implementazione di questo secondo modulo si è basata su scelte ben precise che ho trovato molto interessanti da implementare e che considero un **grosso pilastro del progetto**.

Invece di affidarsi ad **API cloud esterne a pagamento** (come ad esempio Gemini), l'intera pipeline di generazione è stata fatta girare **localmente sul mio portatile** tramite client **Ollama**. Questa scelta ha garantito il controllo totale del processo, l'assenza di costi o limiti di traffico (*rate-limiting*), dimostrando la fattibilità di un sistema di smistamento dati completamente isolato e sicuro.

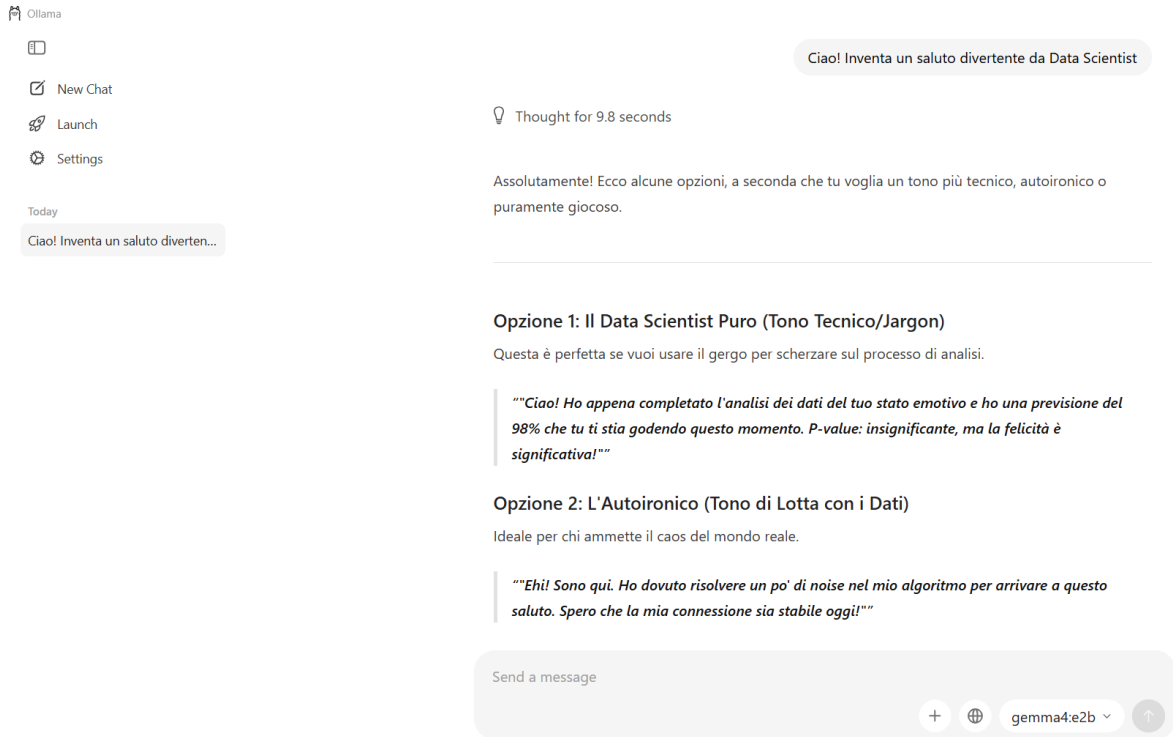


Figura 18: Interfaccia applicazione client di Ollama

Durante le **fasi preliminari di test**, l'utilizzo della versione base di **Gemma4** produce **testi eccellenti** ma richiede **tempi di elaborazione insostenibili**.

Dataset consultabile al seguente link:

https://github.com/gabrieleStagnitta/Project-Work/blob/main/dataset/ticket_gemma4_balanced.csv

[V](#)

body	category	priority
Ciao, sono davvero stufo. Non riesco proprio ad accedere al modulo di invio dati e continua a dare errore. Ho provato riavviare tutto ma non cambia niente. Volete che qualcuno mi dia una mano subito?	Tecnico	Media
Salve, sto riscontrando un comportamento anomalo nell'endpoint /v2/report. Ho notato che i dati relativi al Q3 non vengono popolati correttamente quando li richiamo con il filtro 'client_id=XYZ'. Potrei avere una verifica sulla compatibilità del nuovo schema di richiesta JSON? Grazie.	Commerciale	Media
Buongiorno, devo assolutamente accedere subito all'area riservata perché ho un pagamento da fare oggi. Non riesco proprio a entrare con le credenziali che mi avete dato e il tempo stringe. Potete controllare la mia utenza appena possibile? Grazie.	Amministrazione	Alta
Buongiorno. Ho appena effettuato l'ultimo aggiornamento del software e ora non riesco ad accedere all'area clienti del CRM. Non so cosa sia successo, ma il sistema mi restituisce un errore di autenticazione persistente. Potete aiutarmi a ripristinare l'accesso quanto prima?	Tecnico	Alta
Non so più che fare... Devo mandare queste fatture ma non riesco ad allegarle correttamente al sistema. Ho provato tutto, ho controllato i file e il formato, ma continua a darmi errore. Potete aiutarmi per favore? Sono Marco Rossi.	Amministrazione	Bassa
Buongiorno. Non capisco perché stia succedendo questo casino con la stampante, è un disastro. Ho provato tutto quello che ho trovato su Google, ma non va giù. Credo sia un problema del driver o della rete, e francamente mi sembra una cosa banale per qualcuno di esperto come voi.	Tecnico	Media
Vorrei segnalare che non riesco ad accedere al portale interno per scaricare la documentazione fiscale di questo mese. Ho provato a resettare la password, ma il sistema mi restituisce un errore generico. Potreste aiutarmi con l'accesso urgente? Grazie.	Amministrazione	Alta
Ciao, devo assolutamente accedere a qualche dato cliente urgente. Non riesco proprio a trovare il link giusto o i permessi necessari. Mi puoi aiutare subito con la procedura corretta? Grazie.	Commerciale	Media
Buongiorno! Eccomi di nuovo. Vorrei sapere come posso procedere per verificare l'ultima fattura che ho ricevuto, perché non sono sicuro della voce relativa all'abbonamento annuale. Potreste indicarmi la procedura corretta o se devo contattare un referente specifico? Grazie in anticipo.	Commerciale	Bassa
Ho necessità di segnalare che il portale amministrativo è un disastro. Volete davvero farci passare la notte per ritrovare una semplice fattura? Sono principiante, ma anche voi dovrete sapere dove sono i documenti aziendali. Guardatevi e risolvete subito.	Amministrazione	Alta
Vorrei segnalare che non riesco a accedere alla sezione pagamenti online. Ho provato due volte e mi dà sempre un messaggio di errore generico. Potreste darmi una mano, per favore? Grazie mille!	Amministrazione	Alta
Buongiorno! Eccomi di nuovo. Ho riscontrato un errore nell'autenticazione quando tento di accedere alla sezione relativa all'aggiornamento dei miei dati anagrafici. Potreste verificare la procedura o se ci sono aggiornamenti necessari sul mio account, per favore?	Amministrazione	Bassa
Ho necessità di segnalare un problema con l'integrazione dati. Ho provato a riconfigurare i parametri di connessione manualmente, ma il sistema continua a restituire errori di autenticazione 500. Potrei avere bisogno di una verifica sui permessi lato server o su eventuali dipendenze mancanti per completare la sincronizzazione.	Tecnico	Media

Figura 19: Estratto del dataset generato con Gemma4

La scelta di passare ad un modello più compatto come **Gemma 4:e2b** ha rappresentato il perfetto compromesso: **i tempi si sono ridotti** a oltre 5 ticket al minuto, consentendo la generazione di un

dataset bilanciato di 500 record in meno di due ore e permettendo molteplici sessioni di iterazione sul prompt, al solo costo di avere dei **ticket meno complessi**. Cosa che, come analizzeremo nella prossima sezione, in realtà si è rivelata anch'essa un **punto a favore del progetto**.

Sono estremamente frustrato perché non riesco ad accedere ai dati commerciali necessari per la mia attività. Ho bisogno che questo problema venga risolto immediatamente.	Commerciale	Alta
Ho avuto problemi critici durante l'export dei dati amministrativi, come ho visto con il vecchio sistema concorrente. Mi serve una risoluzione immediata perché devo chiudere le fatture oggi stesso. Per favore, intervenite subito.	Amministrazione	Alta
Buongiorno! Ho riscontrato un piccolo inconveniente nella configurazione della mia rete domestica. Vorrei capire come ottimizzare le impostazioni per una migliore performance generale del sistema.	Tecnico	Bassa
Il disturbo riguarda la latenza durante il trasferimento dati e cerco consigli tecnici su come procedere.		
Ho un problema con il primo utilizzo della piattaforma commerciale. Non riesco ad accedere o a completare le operazioni iniziali come previsto. Potete aiutarmi urgentemente a risolvere questo blocco.	Commerciale	Alta
Da stamattina sto provando a riconnettermi al Wi-Fi ma non riesco a trovare le impostazioni corrette. Ho problemi con il DHCP e non so come fare il reset della scheda di rete. Mi serve aiuto per capire la configurazione base.	Tecnico	Bassa
Da stamattina sto provando a accedere al sistema ma è completamente inattivo. Ho bisogno di una risoluzione immediata per ripristinare il servizio critico.	Tecnico	Alta
La connessione VPN è completamente interrotta. Verifico i log e il tunnel risulta down. Richiedo un intervento immediato per ripristinare la connettività.	Tecnico	Alta
Ho notato che alcune licenze software non sono ancora state sincronizzate con il sistema di gestione commerciale. Vorrei segnalare la situazione per una revisione interna. Grazie.	Commerciale	Bassa
Il modulo di gestione delle fatture è completamente inaccessibile e non riesco a eseguire i report. Questo impatta direttamente le operazioni amministrative e devo risolvere subito.	Amministrazione	Alta
Segnalo una piccolissima anomalia: Ho avuto un piccolo intoppo con la configurazione del mio setup. Non riesco proprio a far funzionare bene il mio vecchio hardware. Mi serve una mano per capire come sistemarlo, se possibile.	Tecnico	Bassa

Figura 20: Estratto del dataset generato con Gemma4:e2b

La classica **sfida nell'utilizzo di qualsiasi LLM** per la creazione di banche dati è l'instabilità dell'output: i modelli tendono infatti ad aggiungere **flussi di coscienza lunghi**, ad aggiungere caratteri in markdown, emoji o **formattazioni impreviste** che creerebbero problemi durante la fase di generazione.

Per eliminare il problema alla radice, il codice integra la libreria **Pydantic** per applicare degli output strutturati e la libreria Ollama per effettuare delle richieste con dei prompt strutturati. Questo permette inoltre di impostare un formato specifico per le risposte del modello (cosa che ha semplificato l'inserimento di ogni singolo ticket nel file .csv di output).

```
class TicketSintetico(BaseModel):
    title: str = Field(
        description="Un titolo breve, specifico e realistico."
    )
    body: str = Field(
        description=(
            "Il corpo del ticket. Deve contenere una descrizione breve (1-2 frasi) del problema reale. "
            "NON usare placeholder, parentesi quadre, simboli di markup o istruzioni di compilazione (es. '[Descrivere qui...]')."
            "Non è obbligatorio inserire nominativi di applicazioni/servizi ma, se lo fai, utilizza nomi verosimili (non usare ad esempio modulo X o endpoint x)"
            "Il testo deve essere testo pronto, discorsivo e già compilato, scritto dal punto di vista dell'utente o del dipendente che sta riscontrando il problema in prima persona"
        )
    )
```

Figura 21: Esempio di struttura base dei parametri di generazione realizzata ereditando la classe BaseModel di Pydantic

```

client = Client()
def genera_ticket_con_gemma(categoria, priorita, esperienza, colloquialita, incipit, titolo, disturbo=False, tipo=False):
    prompt_utente = f"""Genera un ticket di assistenza aziendale in lingua italiana con queste caratteristiche:
    - Categoria del ticket: {categoria}. Questo rappresenta l'argomento principale di cui deve parlare il ticket.
    - Priorità del problema: {priorita}.
    - Livello di esperienza dell'utente: {esperienza}.
    - Livello di colloquialità/tono: {colloquialita}

    STILE DI SCRITTURA:
    1. Sii estremamente sintetico. Il testo deve essere composto da massimo 2-3 frasi corte.
    2. Scrivi in prima persona.
    3. Non usare MAI caratteri speciali di formattazione come asterischi (*), corsivi o grassetto.
    4. Evita deliri o flussi di coscienza lunghi: vai dritto al problema reale.
    5. La forma di saluto iniziale deve essere esattamente "{incipit}" (se non è vuota) o una sua variante naturale.
    6. Usa termini tecnici, acronimi o parole chiave specifiche in base alla categoria del ticket ({categoria}) e all'esperienza ({esperienza}).
    7. Il titolo deve simulare l'oggetto scritto da un utente reale (breve, sbrigativo o imperfetto). Inizia il titolo UTILIZZANDO OBBLIGATORIAMENTE questa esatta espressione in
    8. Rispetta rigorosamente la coerenza: se la priorità è Bassa, non inserire scadenze imminenti o blocchi operativi nel titolo o nel corpo
    """

    if disturbo:
        prompt_utente += """\n - VINCOLO DI CONTESTO DINAMICO (OBBLIGATORIO):
        - {disturbo}. Integra questo dettaglio nel testo in modo naturale, usandolo come elemento di sfondo, senza farlo diventare il problema principale."""

    if tipo:
        prompt_utente += "\n- Inserisci dove rendi opportuno un tipo per rendere il ticket più human like."

    try:
        response = client.chat(
            model='gemma4:2b',
            messages=[{'role': 'user', 'content': prompt_utente}],
            format={
                'type': 'object',
                'properties': {
                    'title': {'type': 'string'},
                    'body': {'type': 'string'}
                },
                'required': ['title', 'body']
            },
            options={
                'keep_alive': -1, # -1 dice a Ollama: "Non scaricare MAI il modello finché non si chiude lo script"
                'temperature': 0.4
            }
        )
        dati_ticket = json.loads(response['message']['content']).strip()
        return dati_ticket['title'], dati_ticket['body']
    except Exception as e:
        print(f"Errore durante la generazione locale: {e}")
        return None, None

```

Figura 22: Funzione della generazione di un ticket. Qui viene formattato il prompt per la creazione di un ticket e sottoposto all'LLM.

Nota: Quelli presenti rappresentano prompt di esempio. I ticket consultabili nei dataset sono stati creati in vari batch, modificando periodicamente le restrizioni in base alle necessità.

La seguente cella di codice sceglie, sorteggiando casualmente da delle liste create precedentemente, tutti i **parametri** da passare per la **generazione del prompt**. L'output di ogni iterazione sarà un singolo ticket, che viene infine conservato all'interno di un .csv.

```

for i in range(0, da_fare):
    id_corrente_numerico = id_di_partenza + i
    stringa_id = f"TK-{id_corrente_numerico}"

    categoria = random.choice(categorie_disponibili)
    priorita = random.choice(priorita_disponibili)
    esperienza = random.choice(livelli_esperienza)
    colloquialita = random.choice(livelli_colloquialita)
    attiva_tipo = random.choices([True, False], weights=[20, 80])[0]

    # si sceglie incipit in base alla priorità, si usa la meccanica dei pesi per creare rumore
    if priorita == "Alta":
        pool_scelta_titolo = random.choices([pool_titolo_urgenti, pool_titolo_normali], weights=[70, 30])[0]
        titolo = random.choice(pool_scelta_titolo)

        pool_scelta_incipit = random.choices([pool_incipit_urgenti, pool_incipit_normali], weights=[70, 30])[0]
        incipit = random.choice(pool_scelta_incipit)

    elif priorita == "Media":
        pool_scelta_titolo = random.choices([pool_titolo_urgenti, pool_titolo_normali, pool_titolo_bassa], weights=[20, 60, 20])[0]
        titolo = random.choice(pool_scelta_titolo)

        pool_scelta_incipit = random.choices([pool_incipit_urgenti, pool_incipit_normali, pool_incipit_bassa], weights=[20, 60, 20])[0]
        incipit = random.choice(pool_scelta_incipit)

    elif priorita == "Bassa":
        pool_scelta_titolo = random.choices([pool_titolo_normali, pool_titolo_bassa], weights=[30, 70])[0]
        titolo = random.choice(pool_scelta_titolo)

        pool_scelta_incipit = random.choices([pool_incipit_normali, pool_incipit_bassa], weights=[30, 70])[0]
        incipit = random.choice(pool_scelta_incipit)

    disturbo = ""
    if random.randint(0,2)==1:
        if categoria == "Tecnico":
            disturbo = f"L'utente specifica di lavorare presso: {random.choice(reparti_aziendali)}"
        elif categoria == "Amministrazione":
            disturbo = f"L'operazione contabile avviene tramite lo strumento: {random.choice(strumenti_generici)}"
        else: # Commerciale
            disturbo = f"L'utente menziona che l'offerta serve per il budget del: {random.choice(reparti_aziendali)} o per lo strumento {random.choice(strumenti_generici)}"

    print(f"Generazione in corso... [{categoria}] - {priorita} - {esperienza} - {colloquialita} - {attiva_tipo} [{(i+1)/(da_fare)}]")
    titolo, corpo = genera_ticket_con_gemma(categoria, priorita, esperienza, colloquialita, incipit, titolo, disturbo, attiva_tipo)
    if titolo and corpo:
        singolo_ticket = {
            "id": [stringa_id],
            "title": [titolo],
            "body": [corpo],
            "category": [categoria],
            "priority": [priorita]
        }
        df_singolo = pd.DataFrame(singolo_ticket)

```

Figura 23: Ciclo responsabile della generazione di n ticket da generare

Pipeline Machine Learning - 03_Pipeline_Machine_Learning.ipynb

In questo Jupyter Notebook viene realizzato il secondo obiettivo del Project Work, ovvero lo sviluppo della pipeline ML. In particolare, è possibile consultare i processi di:

- *Lettura dei dati e EDA*
- *Preprocessing (pulizia base del testo)*
- *Modello classificazione categoria*
- *Modello stima priorità*
- *Valutazione dei modelli*
- *Esportazione dei modelli per la creazione della dashboard*

Come sempre, all'interno del notebook vengono analizzati nel dettaglio **tutti gli elementi utilizzati**.

In questo documento riporto solo un **estratto** dell'**EDA** per il dataset che ha mostrato il **risultato più bilanciato**, ovvero quello realizzato con **Gemma4:e2b**.

Distribuzione dei Ticket per Categoria

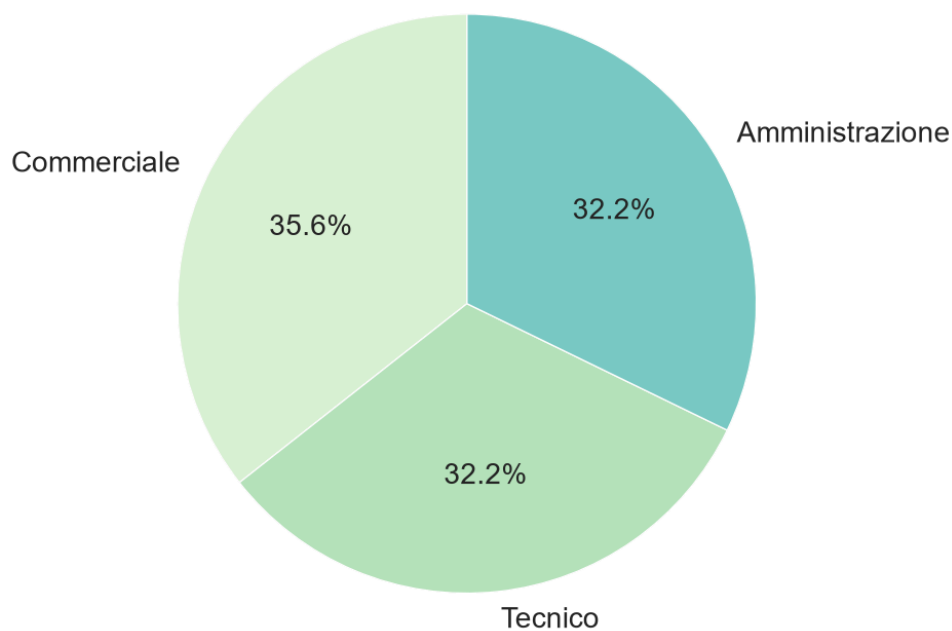


Figura 24.a: Grafico a torta della distribuzione dei ticket per categoria

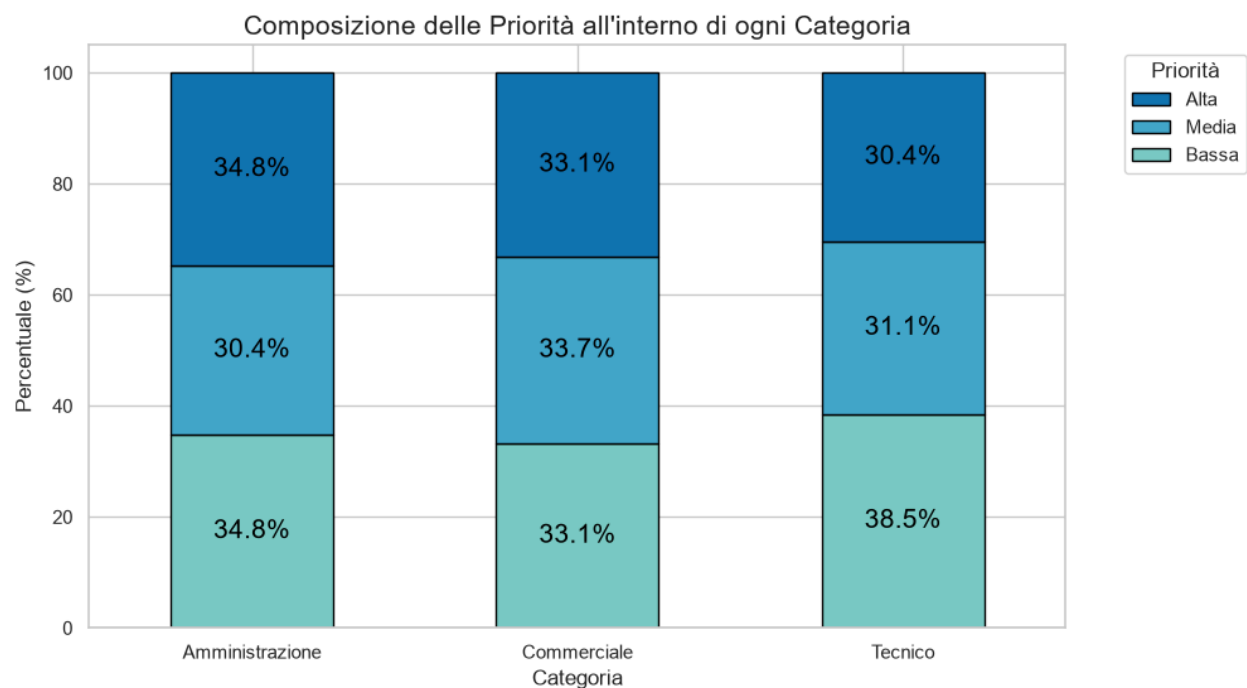


Figura 24.b: Tabella incrociata della composizione priorità nelle varie categorie

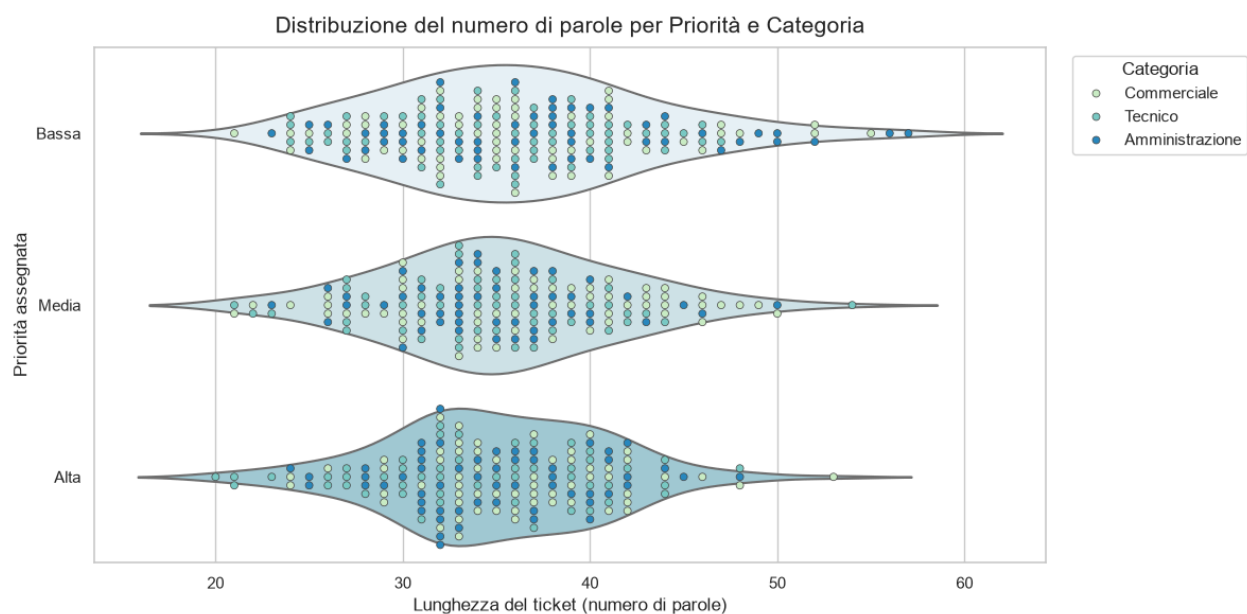


Figura 24.c: Violin plot + Swarm Plot della distribuzione del numero di parole per Priorità e Categoria

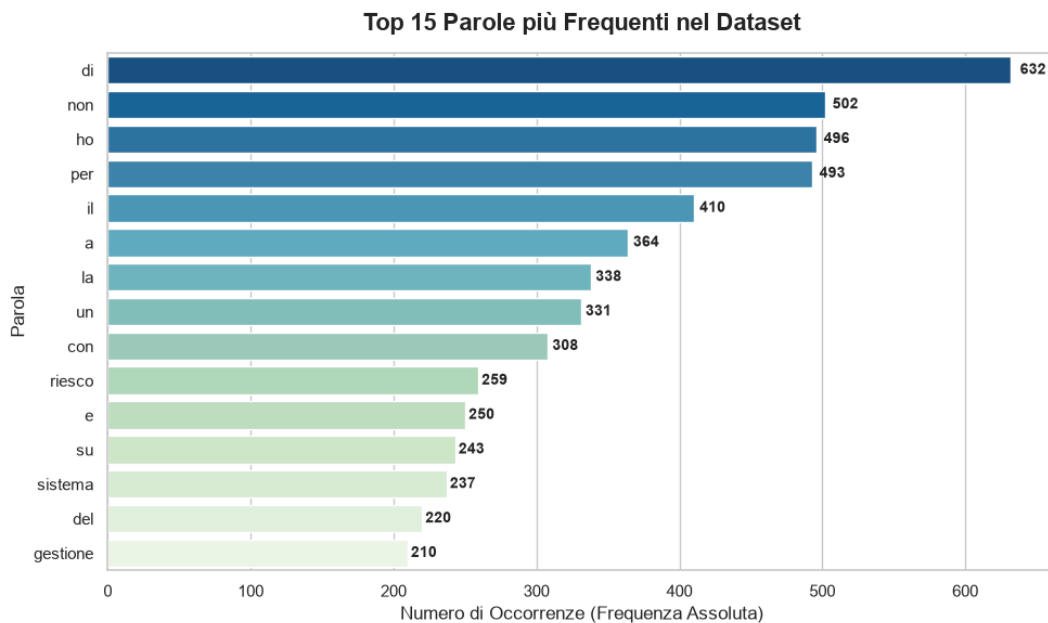


Figura 25: Grafico a barre delle 15 parole più frequenti nel dataset

Come è possibile notare dalle **figure 24.a, 24.b e 23.c**, Gemma4e2b ha rispettato il vincolo di lunghezza dei ticket datogli in quanto, a parte qualche outlier, il numero di parole medio si aggira tra le 30 e le 40 per ticket. Inoltre, si può anche notare come non vi sia un **ottimo bilanciamento della distribuzione** dei ticket sia per categoria, che per priorità.

Come è possibile notare dalla **figura 25**, le parole più utilizzate sono **prettamente stopwords**, ovvero parole che servono a costruire una frase, senza fornire ad essa un significato.

Nonostante questo possa sembrare un "**rumore**", il **vettorizzatore** che verrà presentato a breve **risolve autonomamente questo problema**, dato che il suo compito è quello di assegnare matematicamente un peso a tutte le parole; in particolare, questo abbassa in automatico **il peso delle parole** che compaiono nella maggior parte dei ticket (come appunto le stopwords).

In ogni caso, verrà passato al vettorizzatore una **lista di supporto** contenente stopwords italiane proprio per aiutarlo in questo compito.

```
stop_italiane = ['di', 'a', 'da', 'in', 'con', 'su', 'per', 'tra',
                 'fra', 'il', 'lo', 'la', 'i', 'gli', 'le', 'un', 'uno', 'una',
                 'ho', 'non', 'e', 'del', 'della', 'dei', 'delle']
```

Figura 26: Lista di supporto per le stopwords

Facendo un piccolo passo in avanti, questa operazione **farà scendere leggermente la precisione** nelle previsioni, ma di una percentuale altamente ignorabile. Considerando ciò che abbiamo appena analizzato, questo è un compromesso perfettamente accettabile al fine di **rimuovere il rumore** e fare concentrare gli algoritmi solo sulle **parole realmente importanti** per i ticket.

Ritorniamo indietro e parliamo della procedura di **preprocessing** che si basa prevalentemente su operazioni eseguite utilizzando la libreria **regex**.

La necessità dell'implementazione della funzione `remove_key_word` è maturata a seguito di diverse fasi di **testing fallimentari** con i dataset **realizzati da Gemma**. Ho notato che, nonostante avessi specificato nel prompt di base di non usare parole come “tecnico, commerciale, amministrazione”, queste continuavano ad essere presenti ed essere predominanti nei ticket.

Lo scopo di questa funzione è dunque quello di rimuovere le **radici delle parole** che svelano la **categoria**, ovvero togliere parole come *commerciale*, *amministrazione*, *amministrativo*, ecc, in modo da farle diventare irrilevanti.

```
def remove_key_word(testo):
    testo = testo.lower()
    #rimuove le radici delle parole che svelano la categoria
    testo = re.sub(re.compile(r'\b(tecnic\w*|commerci\w*|amministr\w*|fattur\w*)\b'), '', testo)
    return testo
```

Figura 27: Funzione `remove_key_word`

Superata la fase di **Analisi Esplorativa (EDA)** e ultimata la **pulizia di base** del testo, si è posta la necessità di convertire i ticket in **matrici numeriche**, così da renderle comprensibili per gli **algoritmi di apprendimento supervisionato**. A tale scopo, la pipeline integra il modulo *TfidfVectorizer* della libreria *Scikit-Learn*.

Rispetto a un semplice conteggio delle frequenze, la **misura statistica TF-IDF** permette di pesare **l'importanza di ogni singolo termine**, valorizzando le parole chiave identificative di un reparto (es. "bonifico", "router", "licenza") e **penalizzando i termini generici comuni all'intero dataset** (come le stopword di cui parlavamo prima).

Ho trovato molto interessante la **matematica dietro questo processo**. Senza entrare troppo nel dettaglio, il peso TF-IDF di una parola viene calcolato **moltiplicando la frequenza del termine (TF) per la frequenza inversa del documento (IDF)**, ovvero la misura della rarità di un termine in tutti i documenti. Citando e **semplificando la spiegazione matematica** dell'algoritmo, consultabile da [Wikipedia](https://it.wikipedia.org/wiki/Frequenza_inversa_del_documento):

$$TF(t, d) = \frac{\text{numero di volte che } t \text{ appare in } d}{\text{numero totale di parole in } d}$$

Figura 28.a: Formula del Term Frequency

$$IDF(t) = \ln \left(\frac{\text{numero totale dei documenti}}{\text{numero di documenti che contengono } t} \right)$$

Figura 28.b: Formula dell Inverse Document Frequency

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Figura 28.c: Formula del peso TF-IDF

Dove t rappresenta una **parola**, d il **documento di appartenenza** (in questo caso, un singolo ticket). Per **numero totale dei documenti** in questo progetto si intende invece di numero di ticket totali.

Nota: *TfidfVectorizer* di [Scikit-Learn](https://scikit-learn.org/) implementa una variante leggermente diversa ma più complessa delle formule matematiche, mantenendo però il ragionamento di base.

Per il task di classificazione ho utilizzato **due approcci classici e uno ideato appositamente per il problema**:

- Random Forest Classifier
- LinearSVC
- Approccio Gerarchico

Come ci si può aspettare, **i diversi dataset hanno portato diversi risultati**, per ognuno di essi viene posta un'accurata valutazione dei risultati (guarda la sezione "**Fasi di lavoro...**") per un'analisi più accurata).

Il risultato più stabile per la classificazione delle categorie lo ha dato l'**algoritmo Random Forest** con il dataset `gemma4e2b`.

Per quanto invece riguarda la valutazione della priorità, l'approccio vincente è stato decisamente quello dell'**addestramento gerarchico**. Questa logica consiste nel realizzare inizialmente il modello di classificazione categorie per poi addestrare in maniera verticale i modelli di previsione della Priorità per ogni reparto.

```
#modello per la categorie
vec_cat_gerarchico = TfidfVectorizer(sublinear_tf=True, min_df=5, ngram_range=(1, 2), stop_words=stop_italiane)
X_train_cat_gerarchico = vec_cat_gerarchico.fit_transform(train_df['testo_completo'])
X_test_cat_gerarchico = vec_cat_gerarchico.transform(test_df['testo_completo'])

mod_cat_gerarchico = LinearSVC(random_state=42, dual=False)
mod_cat_gerarchico.fit(X_train_cat_gerarchico, train_df['category'])

#predizione delle categorie (Tecnico, Amministrazione, Commerciale)
test_df['pred_category'] = mod_cat_gerarchico.predict(X_test_cat_gerarchico)

#addestramento modelli di priorità per la categoria
elenco_categorie = train_df['category'].unique()

modelli_priorita_reparto = {}
vectorizers_priorita_reparto = {}

for cat in elenco_categorie:
    #filtra i dati del TRAIN SET per la categoria corrente
    train_sottogruppo = train_df[train_df['category'] == cat]

    vec_prio = TfidfVectorizer(sublinear_tf=True, min_df=2, ngram_range=(1, 2), stop_words=stop_italiane)
    X_train_prio_sub = vec_prio.fit_transform(train_sottogruppo['testo_completo'])

    clf_prio_sub = LinearSVC(random_state=42, dual=False, class_weight='balanced')
    clf_prio_sub.fit(X_train_prio_sub, train_sottogruppo['priority'])

    modelli_priorita_reparto[cat.lower().strip()] = clf_prio_sub
    vectorizers_priorita_reparto[cat.lower().strip()] = vec_prio

#pipeline gerarchico del test set
priorita_predette_finali = []

for idx, riga in test_df.iterrows():
    categoria_stimata = str(riga['pred_category']).lower().strip()
    testo_ticket = riga['testo_completo']

    if categoria_stimata in vectorizers_priorita_reparto:
        vettorizzatore_specifico = vectorizers_priorita_reparto[categoria_stimata]
        modello_specifico = modelli_priorita_reparto[categoria_stimata]

        testo_vettorizzato = vettorizzatore_specifico.transform([testo_ticket])
        prio_stimata = modello_specifico.predict(testo_vettorizzato)[0]
    else:
        prio_stimata = riga['priority']

    priorita_predette_finali.append(prio_stimata)

test_df['pred_priority'] = priorita_predette_finali
```

Figura 29: Approccio gerarchico alla previsione della categoria

Questa cella di codice, nonostante le sue poche righe, ha un **compito molto importante**: salvare i modelli scelti in dei file, che verranno utilizzati successivamente per il funzionamento della dashboard.

```
import joblib

#salviamo i componenti chiave del sistema
joblib.dump(mod_cat_gerarchico, 'modelli/m_categoria.pkl')
joblib.dump(vec_cat_gerarchico, 'modelli/v_categoria.pkl')

#salviamo anche i dizionari dei modelli verticali delle priorità
joblib.dump(modelli_priorita_reparto, 'modelli/m_priorità.pkl')
joblib.dump(vectorizers_priorita_reparto, 'modelli/v_priorità.pkl')

print("Tutti i modelli e i vettorizzatori sono stati serializzati con successo!")
```

Figura 30: Script responsabile al salvataggio di modelli e vettorizzatori

Il notebook si conclude con un'analisi dei dati conclusivi con dei **grafici a barre orizzontali** che mostrano, in base alla categoria o alla priorità, le parole che più influenzano i modelli (il peso delle parole attribuito dai vettorizzatori analizzati precedentemente)

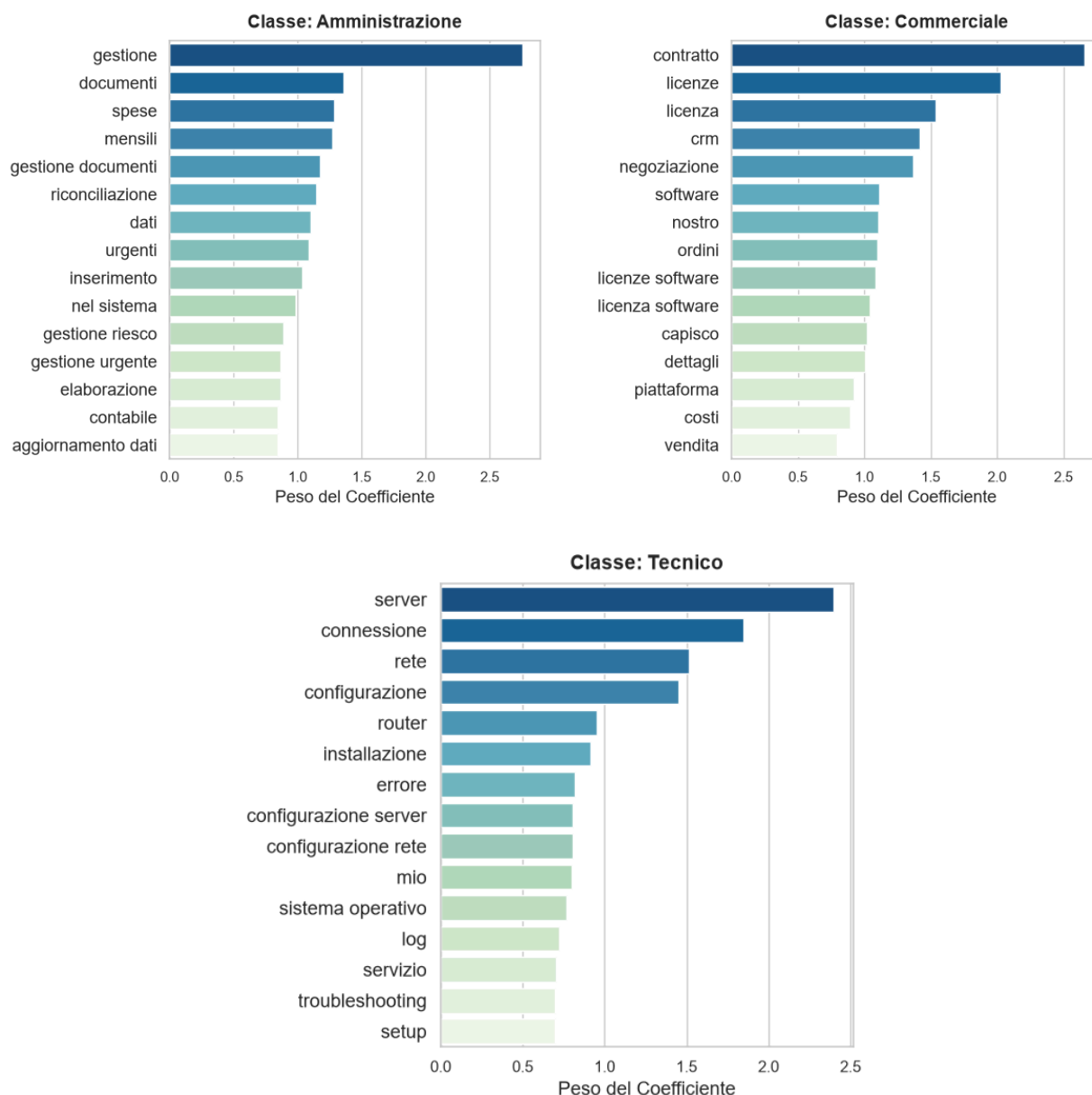


Figura 31.a: Top 15 Parole più influenti per la categoria

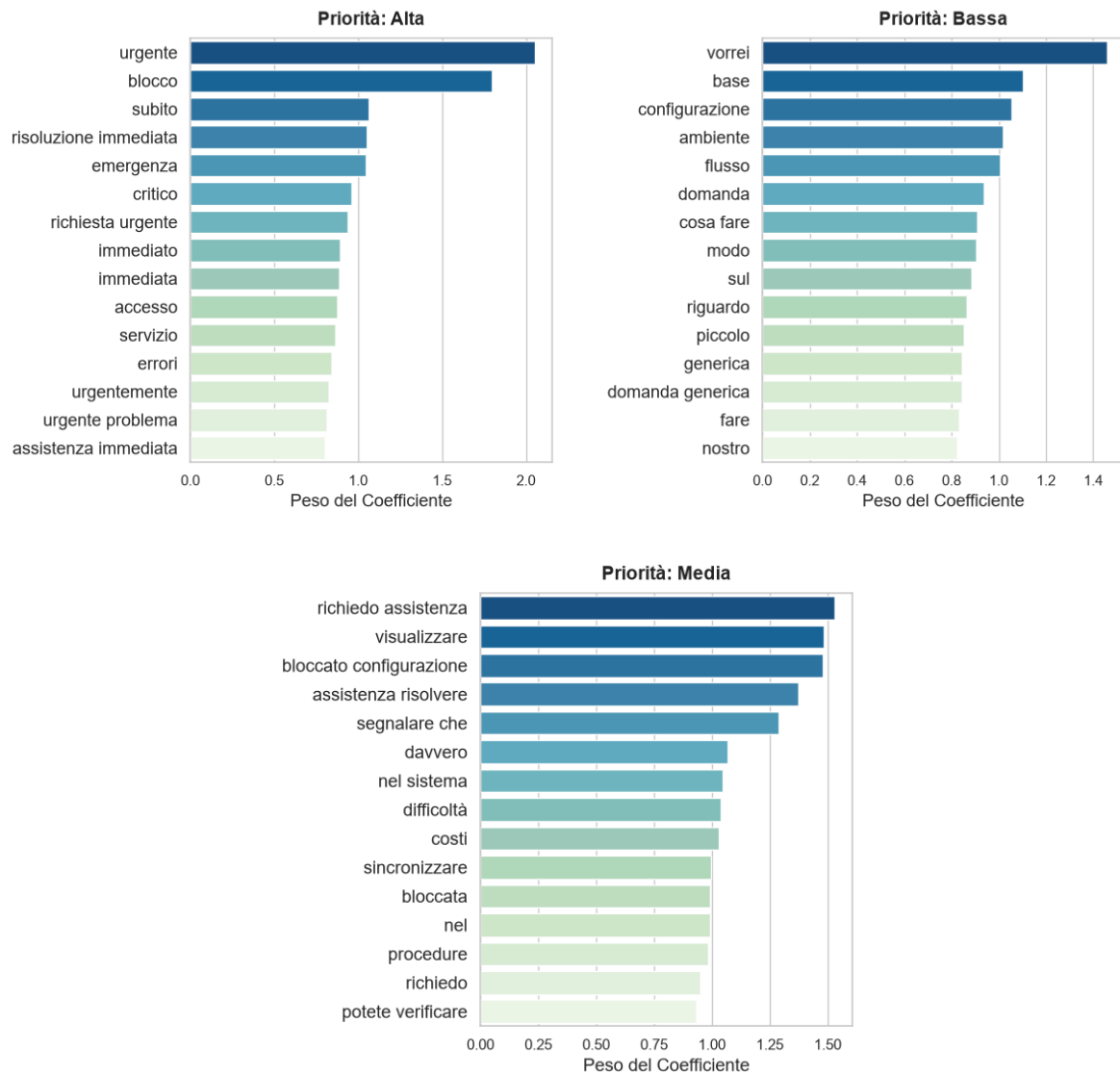


Figura 31.b: Top 15 Parole più influenti per la priorità

Dashboard - [Dashboard.py](#)

In quest'ultimo modulo avviene la trasformazione della pipeline di back-end in un'**interfaccia utente** interattiva, moderna e direttamente fruibile da un **potenziale personale**.

I dati vengono analizzati caricando i modelli e i vectorizer. La funzione *analizza_singolo()* è responsabile dell'analisi di un singolo ticket inserito da un operatore e dell'aggiornamento dell'interfaccia mostrando i risultati dell'analisi.

```
#caricamento modelli
try:
    vec_categoria = joblib.load('modelli/v_categoria.pkl')
    model_cat = joblib.load('modelli/m_categoria.pkl')

    #caricamento dei dizionari per l'approccio gerarchico
    dict_vec_priorita = joblib.load('modelli/v_priorità.pkl')
    dict_mod_priorita = joblib.load('modelli/m_priorità.pkl')
    print("Modelli caricati con successo!")
```

Figura 32.a: Caricamento modelli

```

def analizza_singolo():
    titolo = input_titolo.value
    corpo = input_corpo.value

    if not titolo and not corpo:
        ui.notify('Inserisci testo nel ticket!', type='warning')
        return

    #preprocessing
    testo_pulito = clean_text(titolo + " " + corpo)

    #predizione categoria (fase 1)
    X_cat = vec_categoria.transform([testo_pulito])
    categoria = model_cat.predict(X_cat)[0]

    #chiave di ricerca per il dizionario
    cat_key = str(categoria).lower().strip()

    #predizione priorit  (fase 2 - gerarchica)
    if cat_key in dict_vec_priorita:
        vec_pri_specifico = dict_vec_priorita[cat_key]
        mod_pri_specifico = dict_mod_priorita[cat_key]

        X_pri = vec_pri_specifico.transform([testo_pulito])
        priorit  = mod_pri_specifico.predict(X_pri)[0]

        parole_pri = get_top_words(testo_pulito, vec_pri_specifico, mod_pri_specifico)
    else:
        priorit  = "Sconosciuta"
        parole_pri = []

    #aggiornamento UI
    lbl_categoria.set_text(categoria.upper())
    lbl_priorita.set_text(priorit .upper())

    parole_cat = get_top_words(testo_pulito, vec_categoria, model_cat)

    #aggiorna la lista delle parole influenti con due colori distinti (in base al gruppo di appartenenza)
    container_parole.clear()
    with container_parole:
        for p in parole_cat:
            ui.chip(p, color='primary', text_color='white')

        for p in parole_pri:
            ui.chip(p, color='orange-7', text_color='white')

    ui.notify('Analisi completata!', type='positive')

```

Figura 32.b: Logica dell'analisi del singolo ticket. Le funzioni `clean_text` e `get_top_words` sono le stesse analizzate anche nel notebook 03.

Superati i limiti visivi e strutturali riscontrati inizialmente con librerie desktop tradizionali come Tkinter, lo sviluppo si   indirizzato verso il framework **NiceGUI**. L'applicazione *Dashboard.py*   stata progettata privilegiando un'esperienza d'uso essenziale e priva di attriti cognitivi:

- **Area di testo pulita** in cui l'operatore pu  incollare o digitare il corpo della comunicazione non strutturata.
- **Restituzione istantanea della Categoria prevista e della Priorit  suggerita**
- **Top 5 parole influenti**, che permette di validare o correggere il suggerimento della macchina

Ticket:

Oggetto (Titolo)
Sistema offline e blocco totale

Corpo del Ticket (Descrizione del problema)
Il server principale è andato in crash. Nessuno riesce ad accedere ai gestionali e la produzione è completamente ferma. Serve intervento urgente

ANALIZZA TICKET

Risultato analisi del ticket

Categoria prevista:
TECNICO

Priorità suggerita:
ALTA

Top 5 parole più influenti (categoria e priorità):

server

il server

server principale

principale

il

urgente

blocco

completamente

intervento

accedere

Figura 33: Interfaccia della dashboard - sezione “analisi ticket”

- Area per l’**elaborazione batch** dei ticket fornendo in input un file .csv con all’interno le colonne “title” e “body”.

ANALISI TICKET

ELABORAZIONE BATCH

Carica un file CSV per processare multipli ticket contemporaneamente

IMPORTANTE: Il file deve contenere almeno le colonne “title” e “body”.

Trascina qui il CSV o clicca il tasto +
1.7KB / 100.00%

batch di esempio.csv
1.7KB / 100.00%

Figura 34: Interfaccia della dashboard - sezione “elaborazione batch”

Infine, per soddisfare il requisito di **riproducibilità e rapida esecuzione** dell'elaborato, il codice sorgente della dashboard è stato collegato ad un ulteriore repository GitHub e messo in produzione cloud tramite la piattaforma **Render** al seguente [link](#). Piccolo dettaglio finale a scopo puramente estetico, è possibile visualizzare la web app sia in **light mode** (come nelle figure 33 e 34), che in **dark mode**.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

L'architettura di **Machine Learning applicata allo Smart Ticketing** realizzata nel project work, pur essendo realizzata a partire da un **dataset sintetico e di natura generica**, è stata concepita con una logica modulare. Questa caratteristica ne consente una rapida **adozione e reingegnerizzazione in molteplici contesti organizzativi reali**, ponendosi come strumento di supporto alle decisioni per ottimizzare i flussi di lavoro aziendali.

Ambiti operativi di implementazione:

La **versatilità** della pipeline permette di applicare il modello in diversi settori, **adattando semplicemente il vocabolario del dataset** di addestramento al **dominio di riferimento**:

- **Help Desk IT e Assistenza Tecnica Aziendale:** ambito naturale di applicazione del prototipo. In organizzazioni strutturate (ad esempio società di consulenza informatica), il sistema può smistare istantaneamente le segnalazioni dei dipendenti o dei clienti tra i vari livelli di supporto, assegnando automaticamente una priorità elevata a incidenti critici che bloccano l'operatività.
- **Customer Care:** immaginato **nel mio contesto lavorativo attuale** (scuola online di Coding per bambini), dove giornalmente il reparto di assistenza clienti riceve **comunicazioni** non strutturate sotto forma di email o messaggi Whatsapp da parte dei **genitori** dei nostri studenti.

Il modello potrebbe **catalogare le richieste** senza intervento umano e spedirle **direttamente ai reparti di competenza**, ad esempio *Customer Success* (per info sulle offerte o sui pagamenti), *Supporto Tecnico* (per richieste di aiuto su installazioni e varie) o *Insegnanti* (per informazioni sulle lezioni o su richieste di feedback) o *Sales* (per informazioni riguardo iscrizioni o lezioni di prova).

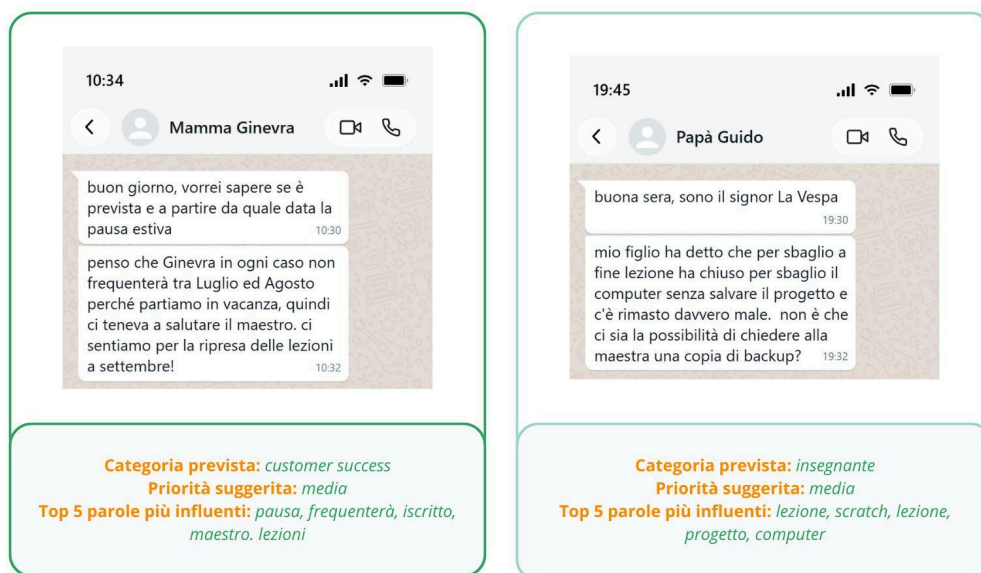


Figura 35: Mockup di un modello di Customer Care, dove sono presenti delle richieste verosimili da parte di genitori immaginari

- **Call Center:** l'architettura potrebbe essere inserita come **primo filtro intelligente** nel mondo dei **call center** di una **qualsiasi tipologia di attività**: dai grandi operatori telefonici alle piccole aziende agricole locali. Abbinando la pipeline di Machine Learning

a un servizio di **Speech-to-Text**, il modello lavorerebbe come **centralino telefonico** intelligente. In questo scenario, è possibile immaginare un cliente **descrivere verbalmente la propria necessità**: il sistema trascrive la **voce in testo**, trasformandolo nell'equivalente dei ticket esaminati nel Project Work. Una volta eseguita l'**analisi della richiesta**, l'algoritmo procede nell'**instradamento in tempo reale** della chiamata all'operatore del **reparto di competenza**, mandando anche una trascrizione della richiesta per anticipare la segnalazione.

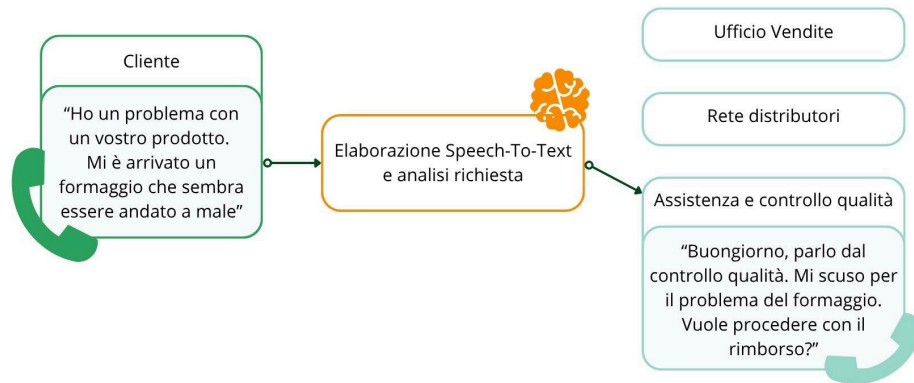


Figura 36: Mockup di un modello di Call Center per un'azienda del settore primario. Le chiamate vengono smistate tra gli operatori in base al reparto di competenza.

Vantaggi strategici e operativi:

L'adozione di un sistema di Triage automatico basato su Machine Learning introduce diversi benefici, tra cui **maggiore competitività** ed **efficienza organizzativa**:

- **Abbattimento del tempo medio di risposta:** l'eliminazione del collo di bottiglia causato dallo smistamento manuale riduce drasticamente il **tempo** che passa tra l'apertura e la presa in carico di un ticket.
- **Riduzione del tasso di errore e/o della soggettività:** lo smistamento manuale rischia di essere influenzato dalle interpretazioni arbitrarie di un problema da parte di un essere umano che spesso non è neanche esperto dell'argomento in questione. Un modello di Machine Learning addestrato invece garantisce *criteri di valutazioni uniformi, oggettivi e replicabili nel tempo*.
- **Ottimizzazione delle risorse e valorizzazione del capitale umano:** questo *non implica togliere lavoro agli esseri umani*, bensì sgravare il personale da mansioni puramente ripetitive e alienanti, dando la possibilità di concentrarsi solo sulla risoluzione dei problemi reali.
- **Trasparenza e supporto all'operatore:** grazie all'integrazione di un software di supporto, come ad esempio una *dashboard interattiva*, il sistema risulta facile da comprendere anche per operatori non specializzati in tutti i settori. L'unico compito sarebbe ricevere un suggerimento e scegliere se accettarlo, mantenendo comunque il **controllo finale** sulla supervisione e sulla validazione del processo.

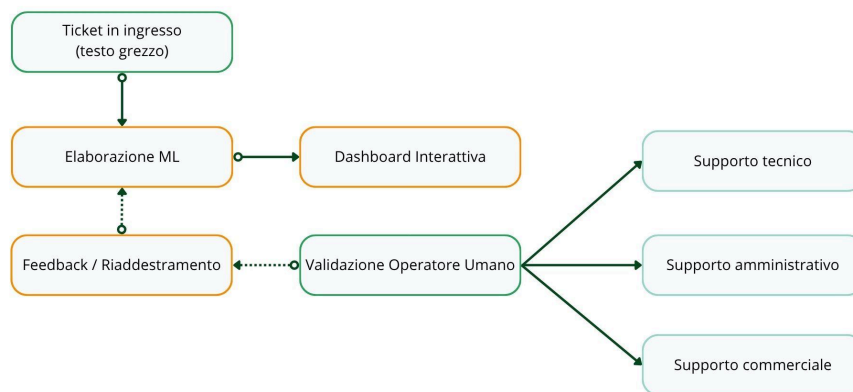


Figura 37: Esempio di flusso di gestione ticket con Machine Learning. In verde i processi umani/preesistenti, in arancione innovazione Machine Learning e in azzurrino output aziendale. Le linee tratteggiate indicano un processo facoltativo che non fa parte del percorso principale.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti)

Concludo l'elaborato con un'analisi critica dei risultati ottenuti da questo prototipo: partiamo dalla generazione dei dati.

L'utilizzo di LLM per la generazione di dataset è un **pilastro importante** del progetto che ne facilita la **scalabilità in molteplici settori**. Nonostante io mi sia soffermato solo sull'analisi della famiglia di modelli Gemma4, in realtà la pipeline è applicabile ad un qualsiasi modello di intelligenza artificiale utilizzabile con Ollama. [Link con vari modelli selezionabili](#). Come ormai tutti sanno, **ogni modello è specializzato** in qualcosa (Claude nel Coding, Perplexity per ricerche approfondite, Gemini per il general purpose, ecc...). Questo implica che si possa scegliere un **modello specifico** in base al contesto in cui ci si trova.

Un limite che però ritengo da considerare è capire quanto la mia analisi sia corretta: il dataset con il modello standard di Gemma4 ha dato risultati scadenti realmente a causa dei **pochi ticket inseriti e alla complessità delle frasi generate?**

Per risolvere questo potenziale problema, oltre all'ovvio tentativo di generare un dataset di dimensione maggiore, ho ragionato su **due potenziali soluzioni**: modifiche dei parametri chiave utilizzati e utilizzo di modelli di Deep Learning avanzati.

Nella pipeline attuale, i **parametri** utilizzati sono abbastanza **generici e standard**, ma potrebbero essere modificati per massimizzare la generalizzazione, ad esempio:

- **Random Forest**: Esplorazione della profondità massima (max_depth) per prevenire l'overfitting sui testi di training, e l'ottimizzazione del numero minimo di campioni per foglia (min_samples_leaf) per migliorare le regole decisionali.
- **LinearSVC**: Variazione del parametro di regolarizzazione (C). Un valore di C ottimizzato potrebbe permettere di trovare il giusto compromesso tra un margine di separazione ampio (che generalizza meglio) e la corretta classificazione dei ticket più ambigui nel training set.

Il secondo approccio consisterebbe nel superamento dell'approccio TF-IDF attraverso l'utilizzo di **BERT** (Bidirectional Encoder Representations from Transformers), un modello di Deep Learning sviluppato da Google e basato sull'architettura **Transformer**. L'integrazione di questa architettura potrebbe garantire due netti miglioramenti:

- **Risoluzione delle ambiguità:** BERT è in grado di comprendere il significato del termine **in base al contesto in cui si trova**. Es: distinguerebbe la frase 'reset del router' dalla frase 'reset del contratto'.
- **Comprensione semantica profonda:** Per sua natura questo modello riesce a riconoscere sinonimi o termini affini (es: 'urgente', 'critico', 'immediato', ecc...). Questo permetterebbe di incrementare non di poco **la precisione sulla classificazione delle priorità**, che è l'area in cui il TF-IDF ha mostrato i **margini di miglioramento più ampi**.

Per quanto riguarda l'interfaccia utente, l'attuale dashboard è un **prototipo** che permette unicamente l'**inserimento manuale dei ticket**, limitandone le potenzialità di utilizzo in un contesto lavorativo reale.

I futuri sviluppi potrebbero concentrarsi sul **collegamento diretto ai canali di comunicazione aziendali** (email o canali di messaggistica), permettendo al software un'**analisi in tempo reale** delle email ricevute, magari affiancandolo con l'aggiunta di un **timer visivo** associato al livello di priorità predetto, che indichi all'operatore il **tempo massimo di presa in carico**.

Infine, sarebbe molto utile permettere all'operatore di **fornire un feedback al modello** (confermando la previsione o correggendo un errore). Questo garantirebbe sempre l'instradamento verso il reparto corretto e, al tempo stesso, consentirebbe di **raccogliere nuovi esempi di ticket** complessi e realistici per alimentare un processo di **apprendimento continuo** (come discusso nella sezione di *campi di applicazione*).